

Universidad Técnica Estatal de Quevedo (UTEQ)

Facultad de Ciencias de la Computación y Diseño Digital

Carrera de Ingeniería de Software

Asignatura: Aplicaciones Web — Período académico 2026–2027

Sistema de Gestión Bibliotecaria Web (SGB-SaaS)

Entrega Final — Informe Académico Completo

Java 21 / Spring Boot 4

Angular 17

PostgreSQL 16

Redis 7

OWASP

Integrantes:

Cajas Ibarra Irvin Marcelo	C.I.: 1251039606	ORCID: 0009-0001-7308-1185
Loor Medranda Marlon Taylor	C.I.: 0928087469	ORCID: 0009-0006-6093-224X
Panama Murillo Moises Antonio	C.I.: 1251334544	ORCID: 0009-0009-8113-242X

Docente-director:

Dr. Gleiston Cicerón Guerrero Ulloa, Ph.D.
gguerrero@uteq.edu.ec

Fecha de cierre:

17 de agosto de 2026

Repositorio:

<https://github.com/mloorm14/sgb-saas> — Rama: main

Tag de esta entrega: v1.0.0

Commit tagueado (completo):
<COMMIT-HASH>

DOI:

<DOI-SOFTWARE-V1.0.0>

SOFT-R-A — 5to Nivel — Proyecto Fin de Curso (PFC), Entrega Final

Índice general

Resumen	5
1. Introducción	6
1.1. Contexto y motivación	6
1.2. Preguntas de investigación	7
1.3. Objetivos	7
1.3.1. Objetivo general	7
1.3.2. Objetivos específicos	7
1.4. Contribuciones	8
1.5. Organización del documento	9
2. Marco teórico	10
2.1. Ingeniería de requisitos: ISO/IEC/IEEE 29148:2018	10
2.2. Características de calidad de requisitos: INCOSE v4	11
2.3. Modelo arquitectónico C4	11
2.4. ISO/IEC 25010: modelo de calidad de producto de software	11
2.5. Principios REST	11
2.6. Autenticación con JWT y transporte seguro	12
2.7. Controles OWASP Top 10:2021	12
2.8. Patrones de acceso a datos: ORM, procedimientos almacenados y <i>cache-aside</i>	13
3. Trabajos relacionados	14
3.1. Estrategia de búsqueda	14
3.2. Proceso de selección	15
3.3. Panorama de trabajos relacionados	16
3.3.1. Sistemas de gestión bibliotecaria: automatización, IA y arquitectura de acceso a datos	16
3.3.2. Ingeniería de requisitos: elicitación y clasificación	22
3.3.3. Prácticas ágiles y efectividad de equipo	22
3.3.4. Diseño responsivo de interfaces web	22
3.3.5. Seguridad de transporte	22
3.3.6. Fundamentos de arquitectura y desarrollo de software	22
3.4. Brecha identificada	23
4. Ingeniería de requisitos	24
4.1. Stakeholders	24
4.2. Técnicas de elicitación	24

4.3.	Los 43 requisitos: categorización MoSCoW	25
4.4.	Proceso de validación: 100 % de los requisitos Must verificados	25
4.5.	Estabilidad del conjunto de requisitos: CHANGELOG-REQ.md	26
4.6.	Checklist de calidad de requisitos (INCOSE v4)	26
5.	Materiales y métodos	28
5.1.	Enfoque metodológico: Design Science Research	28
5.2.	Instrumentos de medición	29
5.3.	Enfoque de métricas: Goal-Question-Metric (GQM)	29
5.4.	Población, muestreo y participantes	30
5.5.	Protocolo experimental de rendimiento (replicable)	31
5.6.	Análisis estadístico	31
5.7.	Determinismo y semillas	32
6.	Diseño y arquitectura del sistema	33
6.1.	Nivel 1 – Contexto del sistema	33
6.2.	Nivel 2 – Contenedores	33
6.3.	Nivel 3 – Componentes	34
6.4.	Decisiones arquitectónicas documentadas (ADR)	35
6.5.	Atributos de calidad (ISO/IEC 25010)	36
6.6.	Matriz de trazabilidad – resumen por tipo de acceso a datos	37
7.	Implementación	39
7.1.	Procedimiento almacenado y su invocación desde Spring Data JPA	39
7.2.	Manejo uniforme de errores (RFC 7807)	43
7.3.	Configuración paramétrica del sistema	44
7.4.	Estrategia de caché Redis del catálogo	45
7.5.	Transporte del token de sesión en cookie HttpOnly	45
8.	Amenazas a la validez	47
8.1.	Validez de constructo	47
8.2.	Validez interna	48
8.3.	Validez externa	48
8.4.	Validez de conclusión	49

Índice de figuras

3.1. Flujo de selección adaptado de PRISMA 2020 para el Grupo A (dominio bibliotecario y arquitectura de acceso a datos comparable). El Grupo B (15 referencias de apoyo metodológico, Sección 3.2) no se somete a este embudo.	16
---	----

Índice de cuadros

3.1. Comparación de trabajos relacionados frente a SGB-SaaS	18
---	----

Resumen

PENDIENTE: se escribe al final, cuando el resto del documento esté completo. El resumen estructurado que exige el Bloque B.2 de la guía (problema, método, resultados principales con cifras reales, conclusión) tiene que sintetizar el documento completo – Introducción, Trabajos Relacionados, Diseño y Arquitectura, Implementación, Resultados, Ingeniería de Requisitos, Amenazas a la Validez, Discusión y Conclusiones – y varios de esos capítulos todavía no existen (ver la nota de estado en `docs/informe-final.tex`). Redactarlo ahora obligaría a inventar resultados y conclusiones que este documento todavía no sostiene, exactamente el tipo de contenido fabricado que este proyecto evita en el resto de su documentación.

Capítulo 1

Introducción

1.1 Contexto y motivación

La gestión de una biblioteca institucional o municipal – control de catálogo, préstamos, devoluciones, reservaciones y multas – se sigue resolviendo con frecuencia mediante procesos manuales o herramientas genéricas no pensadas para el dominio bibliotecario (hojas de cálculo, registros en papel, o sistemas de escritorio aislados sin acceso remoto). En el contexto concreto de una biblioteca universitaria como la de la Universidad Técnica Estatal de Quevedo (UTEQ), esto se traduce en fricción operativa real y verificable en el propio dominio del proyecto: un bibliotecario que registra un préstamo a mano no tiene forma automática de saber si un usuario ya tiene multas pendientes ni de decrementar el stock disponible de forma atómica; un lector no tiene forma de autoservicio para ver sus propios préstamos, reservar un libro o consultar su historial sin acudir físicamente al mostrador.

Nota de honestidad (estimación razonada, no dato de fuente externa verificada). Este documento evita deliberadamente citar una cifra de "población afectada" (número de usuarios de biblioteca, volumen de préstamos anuales, etc.) porque el equipo no cuenta con un estudio o fuente institucional verificada que la respalde – inventar una cifra cuantitativa aquí sería exactamente el tipo de dato fabricado que el resto de este proyecto se niega a producir (ver, por ejemplo, el criterio ya aplicado en `docs/checklists/incose2023-req.md` y en las notas de honestidad de `docs/requisitos/SRS-v1.0.0.md`). Lo que sí se sostiene con evidencia real es el perfil de carga: una biblioteca universitaria como la de UTEQ tiene un volumen de operación bajo y no sostenido en el tiempo (ráfagas puntuales en fechas concretas del calendario académico, no tráfico constante de alta concurrencia), supuesto ya declarado explícitamente en `docs/arquitectura/ISO25010.md` y usado para justificar decisiones de arquitectura (p. ej. la prioridad *Media*, no *Alta*, de la característica "Eficiencia de desempeño").

SGB-SaaS se construye como respuesta a ese problema concreto: una plataforma web (no de escritorio) que digitaliza el ciclo completo de préstamo/devolución/reserva/multa, con control de acceso basado en roles (RBAC) para los distintos perfiles reales de una biblioteca (lector, bibliotecario, gerente, administrador), y que – a diferencia de una implementación mínima que solo "funcione" – se desarro-

lla junto con un programa explícito de evaluación de calidad: pruebas de carga reales, auditoría de seguridad OWASP Top 10 en vivo, ingeniería de requisitos formal con trazabilidad verificable, y un protocolo de usabilidad (SUS) con participantes reales.

1.2 Preguntas de investigación

- **RQ1.** ¿En qué medida el uso de caché (Redis) reduce la latencia observable del endpoint de catálogo bajo carga concurrente controlada, y con qué margen se cumplen los umbrales de rendimiento exigidos?
- **RQ2.** ¿Qué proporción de los controles auditados del OWASP Top 10:2021 presenta hallazgos reales en la implementación, y en qué proporción de esos hallazgos el propio equipo aplicó una corrección verificable dentro del mismo ciclo de desarrollo?
- **RQ3.** ¿Cómo perciben usuarios reales (con roles representativos del dominio, p. ej. lector y bibliotecario) la usabilidad del sistema, medida con el instrumento estandarizado System Usability Scale (SUS)?
- **RQ4.** ¿Qué efecto tiene adoptar una estrategia híbrida de acceso a datos (CRUD elemental vía ORM, operaciones multi-tabla vía procedimientos almacenados) sobre la trazabilidad de requisitos y la cobertura de pruebas automatizadas, frente a un enfoque de solo ORM?

1.3 Objetivos

1.3.1 Objetivo general

Evaluar empíricamente la calidad del sistema SGB-SaaS – en sus dimensiones de rendimiento, seguridad, usabilidad y arquitectura de acceso a datos – mediante evidencia reproducible, versionada y verificable de forma independiente en el propio repositorio del proyecto.

1.3.2 Objetivos específicos

1. **OE1** (trazado a RQ1). Medir, mediante pruebas de carga controladas (k6, 50 VUs concurrentes, múltiples corridas independientes), la latencia p95 del endpoint de catálogo con y sin caché activo, y contrastar el resultado agregado contra los umbrales de aceptación exigidos.
2. **OE2** (trazado a RQ2). Auditar en vivo, contra el stack Docker real (no simulado), un subconjunto representativo de controles del OWASP Top 10:2021, documentando cada hallazgo con evidencia cruda y aplicando corrección verificable a los defectos reales detectados dentro del propio ciclo del proyecto.

3. **OE3** (trazado a RQ3). Aplicar el cuestionario System Usability Scale (SUS) a una muestra de participantes reales, bajo un protocolo de consentimiento informado ya definido, para obtener una puntuación de usabilidad percibida con validez estadística.
4. **OE4** (trazado a RQ4). Documentar y trazar, mediante Architecture Decision Records (ADR) y una matriz de trazabilidad verificada automáticamente en CI, la estrategia híbrida de acceso a datos, cuantificando qué proporción de los requisitos del sistema depende de cada mecanismo (ORM vs. procedimiento almacenado) y qué porcentaje cuenta con prueba automatizada real.
5. **OE5** (transversal a RQ1–RQ4). Construir y versionar toda la evidencia empírica del proyecto – scripts de medición, datos crudos, análisis estadístico – de forma reproducible, para que un tercero pueda regenerarla de forma independiente sin depender de capturas de pantalla ni de afirmaciones sin respaldo.

1.4 Contribuciones

Este trabajo entrega, con evidencia real ya versionada en el repositorio al momento de escribir este capítulo:

- Un sistema web funcional y completo (tag `v1.0.0`) con 43 requisitos especificados y trazados (`docs/trazabilidad/matriz.csv`: 28 funcionales, 15 no funcionales), de los cuales el 100 % de los requisitos de prioridad **Must** cuenta con evidencia de verificación real (ver [Capítulo 4](#)).
- Evidencia empírica reproducible y versionada como código, no como capturas de pantalla editadas a mano: scripts de prueba de carga (`k6/`), análisis estadístico con test pareado de Wilcoxon y tamaño de efecto de Cliff's delta (`scripts/perf-analysis.py`), cobertura de código real (JaCoCo) y auditoría de accesibilidad (Lighthouse).
- Un artefacto de software citable con identificador persistente: el software está archivado en Zenodo con DOI (`10.5281/zenodo.21712467` para la versión `v0.9.0-rc`; la versión `v1.0.0` de este documento se re-archivará al cierre de esta entrega, ver el placeholder de DOI en la portada).
- Un checklist de calidad de requisitos según el marco INCOSE v4 (características C1–C15) aplicado con honestidad metodológica completa: documenta explícitamente qué requisitos no cumplen cada característica y por qué, en vez de reportar cumplimiento universal (`docs/checklists/incose2023-req.md`).
- Documentación arquitectónica versionada como código fuente, no como imágenes estáticas: el modelo C4 completo (niveles 1–3) en Structurizr DSL (`docs/arquitectura/workspace.dsl`) y 13 Architecture Decision Records (`docs/adr/`).

1.5 Organización del documento

El resto de este documento se organiza como sigue. [Capítulo 2](#) sienta las bases conceptuales estrictamente necesarias para el resto del documento – ingeniería de requisitos según ISO/IEC/IEEE 29148:2018, el modelo C4, ISO/IEC 25010, REST, autenticación JWT, OWASP Top 10 y los patrones de acceso a datos (ORM, procedimientos almacenados, *cache-aside*) –, remitiendo a cada capítulo posterior para su aplicación concreta. [Capítulo 3](#) sitúa a SGB-SaaS frente a la literatura académica indexada disponible sobre sistemas de gestión bibliotecaria y los patrones arquitectónicos que el sistema efectivamente utiliza. [Capítulo 4](#) resume el proceso completo de ingeniería de requisitos – stakeholders, técnicas de elicitación, los 43 requisitos categorizados, su validación empírica y su evaluación según el marco de calidad INCOSE v4. [Capítulo 5](#) describe la metodología de *Design Science Research* adoptada, los instrumentos de medición usados (SUS, k6, Lighthouse, JaCoCo, auditoría OWASP), el enfoque de métricas Goal-Question-Metric y el protocolo experimental replicable del bloque de rendimiento. [Capítulo 6](#) describe el diseño arquitectónico del sistema mediante el modelo C4 en sus tres niveles, las decisiones arquitectónicas formalizadas como ADR, y el mapeo contra las características de calidad de producto de ISO/IEC 25010. [Capítulo 7](#) detalla decisiones críticas de implementación con evidencia de código real: la estrategia de configuración paramétrica, el manejo uniforme de errores según RFC 7807, la estrategia de caché y el transporte seguro del token de sesión. [Capítulo 8](#) analiza críticamente las amenazas a la validez de constructo, interna, externa y de conclusión de la evidencia empírica recolectada. Los capítulos de Resultados, Discusión, Conclusiones y Declaraciones/Anexos se incorporarán en tareas de redacción posteriores, una vez completada la evidencia que dependen de ellos (ver la nota de estado en `docs/informe-final.tex`).

Capítulo 2

Marco teórico

Este capítulo sienta únicamente los fundamentos conceptuales **estrictamente necesarios** para entender las decisiones y la evidencia presentadas en el resto del documento – no es un compendio general de cada tema que toca este proyecto. Cuando un concepto ya se desarrolla en profundidad en otro capítulo (la categorización completa de los 43 requisitos, el detalle de las 13 ADR, el código real de cada control OWASP), aquí se presenta solo el marco conceptual mínimo y se remite explícitamente al capítulo que lo aplica.

2.1 Ingeniería de requisitos: ISO/IEC/IEEE 29148:2018

La norma ISO/IEC/IEEE 29148:2018 define el vocabulario y el proceso de referencia para la ingeniería de requisitos de sistemas y software, incluyendo el patrón sintáctico “[condición] [sujeto] *shall* [acción] [objeto] [restricción]” que este proyecto usa como vara de medida en su checklist de calidad (`docs/checklists/incose2023-req.md`, aplicado en detalle en el [Capítulo 4](#)). La elicitación de requisitos – la actividad de descubrir, no solo registrar, lo que un sistema debe hacer – es un proceso empíricamente estudiado y propenso a errores sistemáticos incluso en manos experimentadas: [Bano et al. \(2019\)](#) documentan, a partir de un análisis de errores comunes en entrevistas de elicitación, que problemas como preguntas cerradas prematuras o la proyección de soluciones antes de entender el problema son recurrentes incluso en practicantes entrenados, y [Palomares et al. \(2021\)](#) confirman en un estudio de estado de la práctica en 12 empresas reales que las técnicas de elicitación usadas en la industria rara vez siguen un proceso formal completo, sino una combinación pragmática de entrevistas, prototipado y retroalimentación iterativa – exactamente el patrón que el [Capítulo 4](#) documenta para SGB-SaaS. Este capítulo no repite esa aplicación concreta; solo establece que la elicitación de requisitos es, por naturaleza, un proceso falible que debe verificarse después de ejecutado, no asumirse correcto por haberse seguido un procedimiento nominal.

2.2 Características de calidad de requisitos: INCOSE v4

El marco INCOSE v4 (Guide for Writing Requirements) define nueve características que un requisito individual bien escrito debería cumplir (Necessary, Appropriate, Unambiguous, Complete, Singular, Feasible, Verifiable, Correct, Conforming) y seis características adicionales a nivel de conjunto de requisitos (Complete, Consistent, Feasible, Comprehensible, Able to be validated, Correct). Este capítulo solo nombra el marco como fundamento conceptual; su aplicación completa, requisito por requisito, con hallazgos y excepciones documentadas, está en [docs/checklists/incose2023-req.md](#) y se resume en el [Capítulo 4](#).

2.3 Modelo arquitectónico C4

El modelo C4 ([Brown, 2023](#)) propone documentar la arquitectura de un sistema de software en cuatro niveles de abstracción anidados – Contexto, Contenedores, Componentes y Código – cada uno pensado para una audiencia y un nivel de detalle distintos, en vez de un único diagrama monolítico que intenta servir a todas las audiencias a la vez. SGB-SaaS usa los primeros tres niveles (el nivel de Código se considera cubierto por el propio código fuente versionado). Este capítulo solo introduce el modelo como notación; su aplicación concreta – los tres niveles completos, incluyendo el detalle de los 21 componentes del backend – se desarrolla en el [Capítulo 6](#).

2.4 ISO/IEC 25010: modelo de calidad de producto de software

ISO/IEC 25010:2011 (SQuaRE) organiza la calidad de un producto de software en ocho características de alto nivel – Adecuación funcional, Eficiencia de desempeño, Compatibilidad, Usabilidad, Fiabilidad, Seguridad, Mantenibilidad y Portabilidad – cada una con subcaracterísticas propias, pensadas como un vocabulario común para discutir atributos de calidad no funcionales de forma comparable entre proyectos. Este capítulo solo introduce el marco conceptual; el mapeo completo de SGB-SaaS contra las ocho características, con evidencia y prioridad declarada para cada una, está en [docs/arquitectura/ISO25010.md](#) y se transcribe en el [Capítulo 6](#).

2.5 Principios REST

El estilo arquitectónico REST (*Representational State Transfer*) describe un conjunto de restricciones para servicios web – interfaz uniforme, comunicación cliente-servidor sin estado (*stateless*), cacheabilidad explícita, y direccionamiento de recursos mediante

URIs – que SGB-SaaS adopta para el diseño de su API backend, tal como lo implementa el framework usado en este proyecto (Walls, 2022) mediante controladores anotados (`@RestController`) que exponen recursos (`/api/v1/libros`, `/api/v1/prestamos`, etc.) sobre los verbos HTTP estándar. La restricción de ausencia de estado en el servidor es la que motiva, en particular, que la sesión del usuario autenticado se transporte completa en cada petición (vía el token JWT, Sección 2.6) en vez de mantenerse en memoria del servidor entre peticiones.

2.6 Autenticación con JWT y transporte seguro

JSON Web Token (JWT, RFC 7519) es un formato compacto y auto-contenido para representar afirmaciones (*claims*) entre dos partes, estructurado en tres segmentos separados por puntos – *header*, *payload* y *signature* – codificados en Base64URL y firmados (en el caso de SGB-SaaS, con un secreto simétrico HMAC), de modo que el servidor puede verificar la integridad y autenticidad del token sin necesidad de una consulta a base de datos ni de mantener estado de sesión en memoria – coherente con la restricción *stateless* de REST (Sección 2.5). Este mecanismo no está exento de riesgos de diseño: Bucko et al. (2023) muestran que un esquema JWT ingenuo es vulnerable a robo y reuso de token, y proponen reforzarlo con historial de comportamiento del usuario; Shatnawi et al. (2024) documentan, en un estudio empírico comparativo de librerías JWT en Python, defectos de seguridad concretos (advertencias de *linting* de seguridad) presentes incluso en librerías JWT de uso común; y Ahmed and Mahmood (2019) proponen regenerar el JWT en cada petición del cliente con un componente de marca de tiempo verdaderamente aleatorio para reducir la ventana de reuso de un token robado. Frente a este panorama de riesgos documentados, la decisión de diseño de SGB-SaaS de transportar el JWT en una cookie `HttpOnly` (en vez de en almacenamiento accesible por JavaScript, como `localStorage`) – detallada con código real en el Capítulo 7 – responde directamente a la clase de riesgo que esta literatura describe: un token en una cookie `HttpOnly` no puede leerse ni exfiltrarse mediante una inyección de *script* en el cliente (XSS), que es precisamente el vector de robo de token más citado en la literatura de seguridad de JWT. El transporte de esa cookie sobre una conexión cifrada sigue, a su vez, las guías oficiales de configuración TLS de McKay and Cooper (2019) (NIST SP 800-52 Rev. 2).

2.7 Controles OWASP Top 10:2021

El OWASP Top 10:2021 es una lista de referencia, mantenida por la comunidad de seguridad web OWASP y actualizada periódicamente, de las diez categorías de riesgo de seguridad más críticas y frecuentes en aplicaciones web (control de acceso roto, fallos criptográficos, inyección, mala configuración de seguridad, fallos de identificación y autenticación, fallos de registro y monitoreo, entre otras). Este capítulo solo nombra el marco como referencia; la auditoría real ejecutada contra el stack Docker de SGB-SaaS, control por control, con evidencia reproducible (`curl`) y corrección aplicada a cada hallazgo, está documentada en `docs/mediciones/sec/` y se instrumenta mediante `scripts/owasp-audit.sh` (Capítulo 5).

2.8 Patrones de acceso a datos: ORM, procedimientos almacenados y *cache-aside*

Un *Object-Relational Mapper* (ORM) traduce automáticamente entre el modelo de objetos de un lenguaje orientado a objetos y el modelo relacional de una base de datos, a costa de un control más limitado sobre la sentencia SQL exacta que finalmente se ejecuta. Spring Data JPA, usado en el backend de SGB-SaaS (Walls, 2022), genera esas sentencias automáticamente para operaciones CRUD simples de una sola tabla. Sin embargo, ese mismo automatismo puede degradar el rendimiento en operaciones que involucren múltiples tablas relacionadas o lógica transaccional compleja: Colley et al. (2018) documentan empíricamente, comparando configuraciones de un framework ORM líder contra SQL manualmente optimizado, patrones de rendimiento indeseables (*anti-patrones*) que el uso no cuidadoso de un ORM introduce. Un procedimiento almacenado (*stored procedure*, SP), en contraste, ejecuta lógica SQL directamente dentro del motor de base de datos, con control explícito sobre el plan de ejecución. SGB-SaaS adopta una estrategia híbrida – ORM para CRUD elemental, SP para operaciones multi-tabla transaccionalmente sensibles – cuyo mecanismo concreto de invocación segura (parámetros nombrados bindeados, no concatenación de cadenas) se documenta con código real en el Capítulo 7.

El patrón *cache-aside* (también llamado *lazy loading* de caché) consulta primero una caché de acceso rápido (en este proyecto, Redis) y solo recurre a la base de datos relacional cuando hay un fallo de caché (*cache miss*), momento en el que el resultado se escribe de vuelta en la caché para peticiones futuras. Kusuma et al. (2017) miden empíricamente el efecto de este patrón – separando el tipo de objeto cacheado (contenido estático vía acelerador HTTP, resultados de consulta vía Redis) – documentando reducciones reales de carga sobre la base de datos relacional bajo tráfico concurrente. La instancia concreta de este patrón en SGB-SaaS (anotación `@Cacheable` sobre el catálogo de libros, con invalidación explícita vía `@CacheEvict`) y su evaluación empírica bajo carga (k6, comparación estadística `cache_caliente` vs. `cache_frío`) se desarrollan en el Capítulo 6 y en el Capítulo 5, respectivamente.

Capítulo 3

Trabajos relacionados

Este capítulo sitúa a SGB-SaaS frente a la literatura académica indexada disponible sobre sistemas de gestión bibliotecaria y sobre los patrones arquitectónicos y de proceso que el sistema efectivamente utiliza. Sigue una metodología **adaptada** de PRISMA 2020 – adaptada porque, como se explica en la [Sección 3.1](#), no es una revisión sistemática completa ejecutada desde cero para este capítulo, sino un acervo bibliográfico construido en dos momentos del proyecto y consolidado aquí; se declara esto explícitamente en vez de presentar un embudo de selección más grande o más formal del que realmente se ejecutó.

3.1 Estrategia de búsqueda

Origen del acervo. Las referencias de este capítulo provienen de dos fuentes: (1) un conjunto de 27 referencias curadas por el equipo en fases anteriores del proyecto a partir de búsquedas en IEEE Xplore, ACM Digital Library, Scopus, ScienceDirect (Elsevier) y SpringerLink; y (2) 2 referencias adicionales ([Colley et al. \(2018\)](#) y [Kusuma et al. \(2017\)](#)) identificadas mediante una búsqueda dirigida realizada específicamente para este capítulo – vía Crossref e IEEE Xplore – para cubrir dos vacíos temáticos puntuales que el acervo original no cubría: rendimiento comparado de arquitecturas ORM frente a SQL optimizado, y el patrón *cache-aside* con Redis. Las 30 referencias del archivo `docs/bibliografia.bib` se verificaron una por una contra el registro oficial de Crossref de su DOI antes de transcribirse; esa verificación encontró 5 discrepancias entre los datos originalmente reunidos por el equipo y el registro oficial (autores atribuidos incorrectamente en 3 casos, un DOI que en realidad apunta a un artículo distinto del atribuido en un caso, y un año/autoría equivocados en el caso de la referencia NIST). Cada discrepancia se documenta con su evidencia en el encabezado de `bibliografia.bib` y se usó, en todos los casos, el dato verificado por Crossref.

Cadena de búsqueda de referencia (la usada en las búsquedas originales del equipo y replicada en la búsqueda dirigida de este capítulo):

```
("library management system" OR "biblioteca" OR "library  
automation")  
AND ("QR code" OR RFID OR chatbot OR "recommendation system")
```


OR

"generative AI" OR "stored procedure" OR "cache-aside"

OR Redis OR ORM)

AND (performance OR empirical OR evaluation OR "case study")

Ventana temporal. 2017–2026, coherente con el rango real de publicación de las referencias reunidas (Kusuma et al. (2017), 2017, es la más antigua; Ayinde et al. (2026), 2026, la más reciente).

Criterios de inclusión. (a) publicación arbitrada por pares en una revista indexada (Scopus/JCR) o en actas de una conferencia publicada por IEEE/ACM; (b) publicada entre 2017 y 2026; (c) aborda directamente un sistema de gestión bibliotecaria, o un patrón arquitectónico/de proceso que SGB-SaaS implementa (ORM/SP, caché Redis, IA generativa, elicitación de requisitos, prácticas ágiles, diseño responsivo, TLS); (d) disponible en inglés con resumen consultable.

Criterios de exclusión. Blogs, tutoriales de código sin evaluación empírica, contenido de marketing de proveedores, y fuentes autopublicadas sin arbitraje por pares. Una excepción declarada explícitamente: Brown (2023) (el modelo C4) se cita en este documento como referencia normativa de notación arquitectónica – no como evidencia de investigación empírica – y por eso queda fuera del acervo sometido a cribado de este capítulo.

3.2 Proceso de selección

El acervo de 30 referencias se dividió en dos grupos con tratamiento distinto, porque no todas compiten con SGB-SaaS como sistemas alternativos comparables: solo el primer grupo se sometió al cribado tipo PRISMA de la Figura 3.1.

Grupo A – dominio bibliotecario y arquitectura de acceso a datos comparable (15 referencias). Sistemas o estudios de rendimiento directamente comparables con SGB-SaaS como aplicación: Ayinde et al. (2026); Liabor (2023); More (2024); Rajčić et al. (2025); Supriyono et al. (2018); Ng et al. (2024); Mukund et al. (2021); Adetayo (2023); Yan et al. (2023); Ehrenpreis and DeLooper (2022); Wayesa et al. (2023); Hu and Zhang (2025); Ayemowa et al. (2024); Colley et al. (2018); Kusuma et al. (2017).

Grupo B – literatura de apoyo metodológico y arquitectónico (15 referencias, fuera del cribado PRISMA). Fundamentan decisiones de proceso o de diseño discutidas en otros capítulos, pero no son sistemas de biblioteca ni estudios de rendimiento comparables entre sí: ingeniería de requisitos (Bano et al. (2019); Palomares et al. (2021); Yu (1997); Kurtanović and Maalej (2017)), prácticas ágiles (Truong et al. (2025); Rahman et al. (2024)), diseño responsivo (Parlakkılıç (2022); Aienobe and Iqbal (2025)), seguridad de transporte (McKay and Cooper (2019)), fundamentos de arquitectura de software (Sommerville (2011); Fowler (2002); Walls (2022); Bass et al. (2021); Brown (2023)) y muestreo en investigación empírica de ingeniería de software (Baltes and Ralph (2022), ya citado en el Capítulo 8). No se les aplicó un embudo de selección porque no fueron evaluadas como candidatas alternativas entre sí – se incorporaron directamente por relevancia temática con un capítulo o decisión concreta del proyecto.

Nota de alcance – referencias de JWT/autenticación reservadas para Marco Teórico. Tres referencias adicionales sobre el mecanismo JWT (Bucko et al. (2023); Shatnawi et al. (2024); Ahmed and Mahmood (2019), verificadas contra Crossref con el mismo rigor que el resto del acervo) están en docs/bibliografia.bib pero **no** se desarrollan en este capítulo: no son sistemas bibliotecarios comparables (Grupo A) ni fundamento metodológico general (Grupo B), sino evidencia técnica específica sobre JWT – el mecanismo de autenticación que SGB-SaaS ya implementa (Capítulo 6). Corresponden al capítulo de Marco Teórico (docs/capitulos/04-marco-teorico.tex), que todavía no se ha redactado; se deja esta nota para que, cuando se escriba, incluya esas tres referencias en su discusión de JWT en vez de repetir aquí un análisis que este capítulo no está en condiciones de sostener todavía.

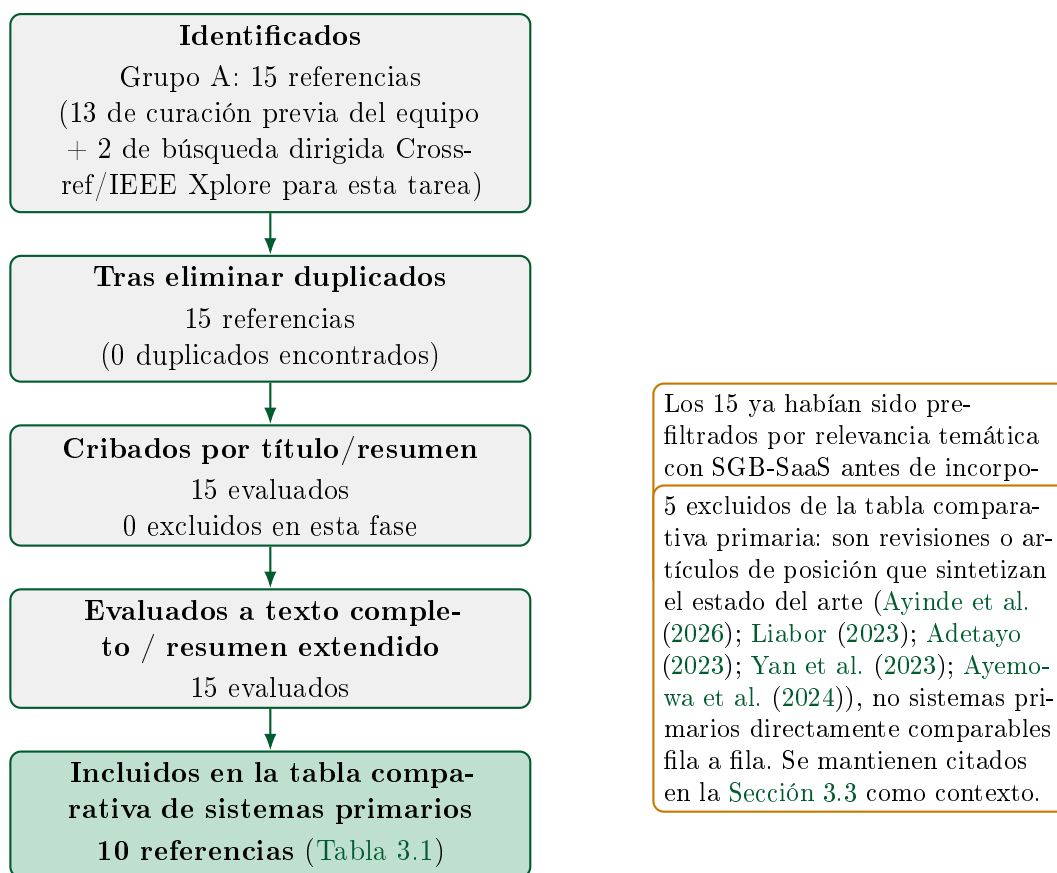


Figura 3.1: Flujo de selección adaptado de PRISMA 2020 para el Grupo A (dominio bibliotecario y arquitectura de acceso a datos comparable). El Grupo B (15 referencias de apoyo metodológico, Sección 3.2) no se somete a este embudo.

3.3 Panorama de trabajos relacionados

3.3.1 Sistemas de gestión bibliotecaria: automatización, IA y arquitectura de acceso a datos

La literatura reciente sobre automatización bibliotecaria confirma una tendencia clara hacia la incorporación de IA y de tecnologías de identificación automática (QR,

RFID), documentada en revisiones amplias como [Ayinde et al. \(2026\)](#) (adopción de IA en bibliotecas académicas) y [Yan et al. \(2023\)](#) / [Ayemowa et al. \(2024\)](#) (revisiones sistemáticas específicas de chatbots y de sistemas de recomendación con IA generativa, respectivamente). [Adetayo \(2023\)](#) documenta, a nivel editorial, la llegada de ChatGPT como punto de inflexión para los chatbots de biblioteca, y [Liabor \(2023\)](#) describe técnicas de catalogación digital sin evaluación empírica cuantitativa propia. Ninguna de estas cinco es un sistema primario evaluable fila a fila contra SGB-SaaS ([Figura 3.1](#)), por lo que se citan aquí como contexto y no aparecen en la tabla comparativa.

La [Tabla 3.1](#) compara SGB-SaaS contra los 10 trabajos primarios seleccionados. Una nota de honestidad metodológica aplica a varias filas: cuando el resumen indexado consultado no permitía confirmar con certeza la pila tecnológica, el método de evaluación empírica o las limitaciones declaradas de un trabajo – por no haberse podido acceder al texto completo del artículo, frecuentemente detrás de un muro de pago editorial – la celda correspondiente dice explícitamente “no confirmado a partir del resumen indexado disponible” en vez de completarse con una suposición razonable pero no verificada.

Cuadro 3.1: Comparación de trabajos relacionados frente a SGB-SaaS

Referencia	Año	Dominio	Pila tecnológica	Patrones arquitectónicos	Evaluación empírica reportada	Limitaciones declaradas	Diferencia frente a SGB-SaaS
Supriyono et al. (2018)	2018	Préstamo/devolución con QR en biblioteca escolar privada (Surakarta, Indonesia)	Web (Bootstrap) + MySQL + lector de cámara USB	Monolito web tradicional, CRUD directo sobre MySQL (no se reporta ORM ni SP)	Pruebas funcionales/de aceptación con usuarios reales de la biblioteca; sin métricas de carga ni estadística formal	Caso único (una escuela); sin datos de escalabilidad	SGB-SaaS usa QR solo como credencial de acceso (REQ-F-019), no como eje central del préstamo, y aporta evidencia de carga formal (k6) ausente aquí
Mukund et al. (2021)	2021	LMS inteligente basado en RFID con analítica predictiva de demanda de libros	RFID + interfaz de escritorio PyQt	Sistema de escritorio con lectura de hardware RFID; no arquitectura web multi-tenant	No confirmado a partir del resumen indexado disponible	No confirmado a partir del resumen indexado disponible; dependencia evidente de hardware RFID especializado	SGB-SaaS es 100 % web multiusuario concurrente sin hardware propietario, con auditoría OWASP y pruebas de carga documentadas
Ng et al. (2024)	2024	Chatbot asistente bibliotecario (Lib-Bot)	No confirmado a partir del resumen indexado disponible	Asistente conversacional dedicado a consultas bibliotecarias	No confirmado a partir del resumen indexado disponible	No confirmado a partir del resumen indexado disponible	SGB-SaaS integra un chatbot con IA generativa real (Gemini, REQ-F-028) para recomendaciones <i>y</i> consultas dentro del mismo sistema transaccional, con <i>rate limiting</i> propio en Redis

Referencia	Año	Dominio	Pila tecnológica	Patrones arquitectónicos	Evaluación empírica reportada	Limitaciones declaradas	Diferencia frente a SGB-SaaS
Rajačić et al. (2025)	2025	Validación de cumplimiento de GDPR en un LMS real (BISIS, +60 bibliotecas en Serbia) integrando el framework TeMDA	BISIS (LMS en producción) + TeMDA (framework de validación dirigido por modelos)	Integración de un framework externo de validación de privacidad sobre un LMS ya productivo; caso de estudio con diario de desarrollo	Estudio de caso cualitativo (diario de desarrollo); sin métricas cuantitativas de rendimiento ni pruebas de carga	Caso único, cualitativo, sin generalización estadística	SGB-SaaS no implementa un framework dedicado de validación GDPR – gap real, no solo diferencia –; solo cuenta con bitácora de auditoría interna (REQ-F-003) sin verificación normativa formal
Wayesa et al. (2023)	2023	Recomendación híbrida de libros basada en patrones y relaciones semánticas	No confirmado en detalle a partir del resumen indexado disponible	Módulo de recomendación independiente, no integrado a un LMS transaccional completo	Evaluación de calidad de recomendación (según título/resumen); detalle cuantitativo no confirmado en el resumen consultado	No confirmado en detalle a partir del resumen indexado disponible	SGB-SaaS no implementa un modelo de recomendación semántica dedicado; delega la recomendación a un chatbot con IA generativa de propósito general (Gemini) – <i>trade-off</i> explícito, no superioridad
Hu and Zhang (2025)	2025	Analítica de big data e IA en servicios de biblioteca: recomendación personalizada y optimización de catálogo	Framework de IA con autoencoder variacional (VAE) y optimizador Adam (según fragmento de resumen disponible)	Pipeline de analítica/ML sobre datos de catálogo, orientado a optimización, no a transacciones de préstamo	No confirmado en detalle – acceso completo restringido por muro de pago editorial (IEEE Access)	No confirmado en detalle (mismo motivo)	SGB-SaaS no entrena un modelo de ML propio (VAE ni similar); delega la personalización a una API de IA generativa externa – arquitectura más simple, dependiente de proveedor externo

Referencia	Año	Dominio	Pila tecnológica	Patrones arquitectónicos	Evaluación empírica reportada	Limitaciones declaradas	Diferencia frente a SGB-SaaS
Ehrenpreis and DeLooper (2022)	2022	Implementación de un chatbot propietario (Ivy) en el sitio web de una biblioteca universitaria (Lehman College) – primer caso documentado en biblioteca académica	Software de chatbot educativo propietario embebido en sitio web institucional	Chatbot de terceros integrado solo al sitio informativo, sin conexión a las transacciones del sistema de préstamos	Estudio de caso descriptivo de implementación; sin métricas cuantitativas de precisión ni de carga	Descripción de un solo caso; sin datos cuantitativos de efectividad del chatbot	El chatbot de SGB-SaaS no es un producto de terceros aislado: es un módulo (REQ-F-028) integrado a los datos transaccionales reales del catálogo
More (2024)	2024	Sistema inteligente de gestión bibliotecaria con RFID, módulo GSM y microcontrolador AVR	RFID + GSM + microcontrolador AVR + Visual Basic (según palabras clave del resumen indexado)	Sistema embebido/de escritorio orientado a automatización física (notificación por SMS vía GSM), no arquitectura SaaS web	No confirmado en detalle a partir del resumen indexado disponible	No confirmado en detalle (mismo motivo); dependencia de hardware especializado evidente por el propio diseño	SGB-SaaS es una plataforma SaaS 100 % software, desplegable en cualquier navegador, con notificación por correo (SMTP) en vez de SMS/GSM
Colley et al. (2018)	2018	Impacto de frameworks ORM en el rendimiento de consultas relacionales (estudio general de acceso a datos, no específico de bibliotecas)	Stack de aplicación Java líder de mercado (no se especifica dominio bibliotecario)	Identifica patrones de rendimiento indeseables (anti-patrones) producidos por el uso de ORM frente a SQL optimizado	Comparación empírica real de tiempos de ejecución y uso de memoria entre configuraciones ORM	No evalúa explícitamente una arquitectura híbrida ORM+procedimientos almacenados como estrategia de mitigación	SGB-SaaS sí implementa y mide empíricamente (k6, Wilcoxon) la estrategia híbrida CRUD-ORM/SP que este trabajo identifica como necesaria pero no llega a ejecutar ni medir (Sección 7.1)

Referencia	Año	Dominio	Pila tecnológica	Patrones arquitectónicos	Evaluación empírica reportada	Limitaciones declaradas	Diferencia frente a SGB-SaaS
Kusuma et al. (2017)	2017	Comparación de estrategias de caché (acelerador HTTP + Redis) en WordPress multisitio – no bibliotecario	WordPress + Varnish (caché HTTP) + Redis (caché de objetos) + MySQL	<i>Cache-aside</i> separado por tipo de objeto: estático vía Varnish, consultas de BD vía Redis	Medición real de uso de CPU de MySQL (−25 %) y tráfico de red (−94 %) bajo carga, con caché Redis activo	Contexto específico de WordPress multisitio; sin test estadístico formal (<i>p</i> -valor, tamaño de efecto)	SGB-SaaS usa un patrón <i>cache-aside</i> equivalente (Sección 7.4) sobre una API REST propia, con evaluación estadística formal (Wilcoxon, Cliff's delta) que este trabajo no reporta

3.3.2 Ingeniería de requisitos: elicitación y clasificación

La calidad del proceso de elicitación de requisitos es un tema activo de investigación empírica: [Bano et al. \(2019\)](#) estudian, mediante un análisis de errores comunes, cómo se enseña y se aprende a conducir entrevistas de elicitación, y [Palomares et al. \(2021\)](#) amplían el panorama con un estudio de estado de la práctica en 12 empresas reales. [Yu \(1997\)](#) aporta el marco conceptual clásico (i^*) para razonar sobre requisitos en fase temprana en términos de actores e intenciones, y [Kurtanović and Maalej \(2017\)](#) atacan un problema técnico directamente relevante para este proyecto: la clasificación automática de requisitos funcionales frente a no funcionales mediante aprendizaje supervisado – la misma distinción F/NF que estructura la matriz de trazabilidad de SGB-SaaS ([Capítulo 4](#)), aunque en este proyecto esa clasificación se hizo manualmente y no con un clasificador entrenado.

3.3.3 Prácticas ágiles y efectividad de equipo

[Truong et al. \(2025\)](#) encuentran, en equipos Scrum de empresas vietnamitas, una asociación medible entre rasgos de personalidad del equipo y su efectividad percibida, y [Rahman et al. \(2024\)](#) reportan evidencia de que la adopción de prácticas ágiles impacta la productividad de equipos de desarrollo. Ambos trabajos son relevantes como marco de referencia para contextualizar (no medir formalmente, dado que está fuera del alcance de este proyecto) el propio proceso de un equipo de 3 personas sin dedicación exclusiva que construyó SGB-SaaS ([Capítulo 8](#)).

3.3.4 Diseño responsivo de interfaces web

[Parlakkılıç \(2022\)](#) evalúan el efecto del diseño responsivo sobre la usabilidad de sitios web académicos durante la pandemia, y [Aienobe and Iqbal \(2025\)](#) revisan patrones de diseño responsivo orientados a mejorar UX/UI. Ambos fundamentan la decisión de construir el frontend de SGB-SaaS como una aplicación Angular responsiva – decisión cuya validación de usabilidad real (SUS) sigue pendiente al momento de escribir este documento ([Sección 8.3](#)).

3.3.5 Seguridad de transporte

[McKay and Cooper \(2019\)](#) (NIST SP 800-52 Rev. 2) documentan las guías oficiales para la selección y configuración de implementaciones TLS, la referencia normativa detrás de las decisiones de transporte seguro de credenciales de SGB-SaaS (cookies `HttpOnly`, [Capítulo 7](#)).

3.3.6 Fundamentos de arquitectura y desarrollo de software

Las decisiones de diseño documentadas en el [Capítulo 6](#) y el [Capítulo 7](#) se apoyan en literatura fundacional consolidada: [Sommerville \(2011\)](#) para el proceso de ingeniería

de software en general, Fowler (2002) para los patrones de arquitectura de aplicaciones empresariales (incluyendo el patrón de acceso a datos que motiva la estrategia híbrida ORM/SP), Walls (2022) para el framework Spring usado en el backend, Bass et al. (2021) para el vocabulario de atributos de calidad usado en el mapeo a ISO/IEC 25010 (Capítulo 6), y Brown (2023) para la notación C4 usada para documentar la arquitectura – citado, como se aclaró en la Sección 3.1, como referencia técnica normativa y no como evidencia empírica de investigación.

3.4 Brecha identificada

Ningún trabajo revisado en este capítulo integra, en un solo sistema, las cuatro características que SGB-SaaS combina: (i) autenticación basada en JWT con control de acceso por roles (RBAC); (ii) un catálogo con autocompletado de ISBN; (iii) un chatbot con IA generativa real –no un motor de intents fijos– usado simultáneamente para recomendaciones y para consultas sobre el catálogo (REQ-F-028); y (iv) una arquitectura híbrida de acceso a datos (CRUD elemental vía ORM, operaciones multi-tabla vía procedimientos almacenados) respaldada por evidencia empírica de rendimiento reproducible (k6, 5 corridas independientes, prueba de Wilcoxon pareada, tamaño de efecto de Cliff’s delta).

De los 10 trabajos primarios comparados, los más cercanos a SGB-SaaS en cada dimensión individual son: Ng et al. (2024) y Ehrenpreis and DeLooper (2022) en la dimensión de chatbot (pero ninguno de los dos reporta el uso de un modelo de IA generativa de propósito general integrado a las transacciones del sistema, a diferencia de Gemini en SGB-SaaS); Colley et al. (2018) en la dimensión de arquitectura de acceso a datos (identifica el problema que la estrategia híbrida de SGB-SaaS resuelve, pero no implementa ni mide esa estrategia); y Kusuma et al. (2017) en la dimensión de caché (mide el efecto de Redis, pero sin la evaluación estadística formal que docs/mediciones/perf/REPORT.md aporta para SGB-SaaS). Ningún trabajo del acervo revisado combina las cuatro dimensiones en un solo sistema evaluado empíricamente.

Esta afirmación de brecha se hace con dos advertencias explícitas de honestidad metodológica, coherentes con el resto de este documento: primera, está acotada estrictamente a las 30 referencias efectivamente revisadas en este capítulo (Sección 3.1) – no es una afirmación sobre la totalidad de la literatura publicada sobre sistemas de gestión bibliotecaria, que es considerablemente más extensa que el acervo aquí reunido; segunda, la brecha se refiere a la *combinación* de las cuatro características, no a la novedad de cada característica individual, varias de las cuales (chatbots, recomendación, RFID, QR) están bien establecidas por separado en la literatura revisada. Esta misma limitación de alcance de la revisión se retoma como amenaza a la validez externa en el Capítulo 8.

Capítulo 4

Ingeniería de requisitos

4.1 Stakeholders

El proyecto identifica cinco grupos de interesados reales, no hipotéticos: los cuatro roles de usuario final del sistema – **Lector** (estudiante o miembro de la comunidad universitaria), **Bibliotecario** (personal de mostrador, bajo conocimiento técnico esperado), **Gerente** (responsable de la biblioteca, además de las funciones de Bibliotecario puede anular multas y ver reportes) y **Administrador** (administrador técnico/institucional del sistema) – más el **docente-director** (Dr. Gleiston Cicerón Guerrero Ulloa, Ph.D.) como evaluador académico del PFC y fuente directa de retroalimentación formal (ver las observaciones de Entrega 1A, Entrega 1B y Tercera Entrega en `docs/observaciones/OBSERVACIONES.md`). El propio equipo de desarrollo (3 integrantes) actúa simultáneamente como responsable de elicitación, implementación y verificación – una restricción real de recursos declarada explícitamente en `docs/requisitos/SRS-v1.0.0.md` (sección 2.4) que explica, entre otras cosas, por qué ciertos requisitos quedan con evidencia parcial en vez de simularse como completos.

4.2 Técnicas de elicitación

El proyecto combina tres fuentes de requisitos, ninguna ficticia:

- **Historias de usuario y casos de uso**, en formato Connextra (“Como *rol*, quiero *acción*, para *beneficio*”) y Cockburn respectivamente, versionados como un archivo por HU/CU en `docs/requisitos/historias/` y `docs/requisitos/casos-de-uso/` para el módulo original del equipo, y consolidados en dos archivos únicos (`historias-usuario.md`, `casos-de-uso.md`) para las 5 HU/CU del módulo de Préstamos/Reservas/Multas – una inconsistencia real de convención entre módulos, documentada explícitamente en el SRS en vez de normalizada en silencio.
- **Hallazgos de auditoría de seguridad como fuente directa de requisitos no funcionales**: REQ-NF-006 (*rate limiting* de login) y REQ-NF-007 (auditoría de eventos de autenticación) no eran requisitos preexistentes – nacieron de un

hallazgo real de la auditoría OWASP A07/A09 (ausencia total de límite de intentos y de registro de eventos), corregido dentro del mismo ciclo de desarrollo.

- **Inspección directa del código como último recurso declarado**, únicamente para los módulos construidos después de la Tercera Entrega que no llegaron a tener HU/CU dedicada – nunca presentada como si una HU real existiera. De los 43 requisitos, 8 (REQ-F-010, REQ-F-020 a REQ-F-022, REQ-F-025 a REQ-F-028) no citan ninguna HU/CU en la matriz de trazabilidad, y otros 5 (REQ-F-017, REQ-F-018, REQ-F-019 parcialmente, REQ-F-023, REQ-F-024) citan un identificador de HU/CU que **no corresponde a ningún archivo real** del repositorio, verificado por búsqueda exhaustiva antes de escribir el SRS – en ambos casos, el rationale de esos requisitos en el SRS se declara explícitamente como inferido de la implementación, no como un hecho documentado previamente.

4.3 Los 43 requisitos: categorización MoSCoW

La fuente única de verdad es `docs/trazabilidad/matriz.csv`, validada automáticamente en cada ejecución de CI por `scripts/validate-traceability.sh`. Este capítulo resume su distribución agregada; el detalle de cada requisito individual (descripción, rationale, criterio de aceptación medible, método de verificación) vive en `docs/requisitos/SRS-v1.0.0.md` y en la propia matriz – no se reproducen aquí las 43 fichas completas.

Dimensión	Categoría	Cantidad (%)
Tipo	Funcional (REQ-F)	28 (65,1 %)
	No funcional (REQ-NF)	15 (34,9 %)
Prioridad MoSCoW	Must	23 (53,5 %)
	Should	20 (46,5 %)
	Could	0 (0,0 %)
	Won't	0 (0,0 %)
Estado (matriz)	verificado	23 (53,5 %)
	implementado	20 (46,5 %)
Total		43 (100,0 %)

Ningún requisito usa las categorías **Could** o **Won't** de MoSCoW – dato real, no una omisión de esta tabla: los 43 requisitos del sistema se consideraron, al momento de incorporarse, o bien imprescindibles (**Must**) o bien deseables dentro del alcance actual (**Should**); ninguno se registró formalmente como descartado o como candidato de prioridad baja.

4.4 Proceso de validación: 100 % de los requisitos Must verificados

Al cierre de este capítulo, los 23 requisitos de prioridad **Must** tienen estado **verificado** en la matriz – es decir, cuentan con prueba automatizada real y/o evidencia empírica directa contra el stack en ejecución, no solo con código que compila. Este 100 % es

reciente: los dos últimos requisitos **Must** en estado **implementado** (REQ-F-001, registro de usuario, y REQ-NF-013, prevención de inyección SQL) carecían de prueba automatizada de regresión para parte de su criterio de aceptación – REQ-F-001 solo tenía cubierto el criterio de rechazo por correo duplicado, no el flujo exitoso ni el rechazo por contraseña corta; REQ-NF-013 solo tenía verificación manual puntual de la auditoría original, sin test de regresión permanente. Se cerraron agregando 2 tests nuevos (`AuthControllerTest.registro_passwordCorta_devuelve400ProblemDetail` y `AuthServiceTest.registroConPayloadDeInyeccionSql_seGuardaComoTextoLiteralSinLanzarExcepcion`, este último una regresión permanente del payload de inyección SQL verificado manualmente en la auditoría OWASP A03 original) en el commit `e1f0c25`.

4.5 Estabilidad del conjunto de requisitos: CHANGELOG-REQ.md

`docs/requisitos/CHANGELOG-REQ.md` compara el conjunto de requisitos entre la Tercera Entrega (`docs/requisitos/historico/SRS-v0.9.0-rc.md`, 30 requisitos) y esta entrega (`SRS-v1.0.0.md`, 43 requisitos), comparando el **cuerpo completo** de cada requisito compartido, no solo su título – metodología que descartó explícitamente 2 falsos positivos (REQ-F-016, REQ-NF-009) que una primera versión del script de comparación marcó como “modificados” por un artefacto de extracción de texto (un separador Markdown capturado como parte del cuerpo), documentado en el propio `CHANGELOG-REQ.md` en vez de contar una modificación que en realidad no ocurrió. El resultado: 13 requisitos agregados, 2 genuinamente modificados (REQ-NF-012/TLS y REQ-NF-014/CSP, ambos reclasificados de “pendiente” a “parcialmente implementado” tras cerrar parte de su alcance), 0 eliminados – **tasa de estabilidad** = $1 - (2/43) = 0,9535 \approx 0,953$, es decir, **95,3 %**.

Nota de honestidad (hallazgo de este capítulo, no de CHANGELOG-REQ.md). Ese mismo archivo reporta también un “porcentaje verificado” de 21/43, es decir 48,8 %, calculado en el momento en que se escribió. Esa cifra quedó desactualizada por el propio cierre descrito en la sección anterior: tras el commit `e1f0c25` (posterior a `CHANGELOG-REQ.md`), el conteo real de requisitos **verificado** es 23/43, es decir 53,5 %, no 48,8 %. Se señala aquí en vez de propagar silenciosamente la cifra desactualizada de la fuente citada – `CHANGELOG-REQ.md` en sí no se corrige en esta tarea, por estar fuera de su alcance.

4.6 Checklist de calidad de requisitos (INCOSE v4)

`docs/checklists/incose2023-req.md` evalúa los 43 requisitos contra las 9 características individuales (C1–C9) del marco INCOSE v4 y las 6 características de conjunto (C10–C15), con el mismo criterio de honestidad metodológica del resto del proyecto: no se marca cumplimiento por defecto.

Resultado agregado, considerando solo las 8 características de contenido

(C1–C8): **20 de 43 requisitos (46,5 %)** pasan las 8 sin ninguna excepción; los 23 restantes (53,5 %) tienen al menos una excepción documentada con evidencia concreta – típicamente C5 (*Singular*: un requisito agrupa más de una acción o regla de negocio distinguible bajo un mismo id, 12 casos) o C2 (*Appropriate*: criterio de aceptación demasiado escueto frente a requisitos hermanos del mismo módulo, 5 casos); las de C8 (*Correct*) son las menos frecuentes (4 casos) pero las más específicas, incluyendo una que este mismo checklist encontró antes de que se cerrara: la afirmación de REQ-F-001/REQ-NF-013 de que ciertos criterios carecían de prueba automatizada, ya resuelta en el commit `e1f0c25` citado en la sección anterior.

Hallazgo sistémico de C9 (*Conforming*), declarado sin ambigüedad: ninguno de los 43 requisitos está redactado como una única cláusula “[condición] [sujeto] *shall* [acción] [objeto] [restricción]” del patrón formal de ISO/IEC/IEEE 29148:2018. El formato real del SRS separa *Descripción* (sujeto + modal + acción + objeto, en prosa) de *Criterio de aceptación medible* (las condiciones/restricciones, en bullets aparte) – un híbrido Volere/Cockburn/Gherkin, no el patrón EARS de una sola oración. El propio checklist documenta esto como una desviación **estructural y sistemática**, no un defecto puntual de contenido de alguno de los 43 – razón por la cual, si se cuentan las 9 características en vez de solo las 8 de contenido, el resultado formal es 0 de 43 requisitos sin ninguna excepción (100 % con al menos una), enteramente explicado por este hallazgo de formato, no por 43 fallas de contenido independientes. No ocultar este resultado – aunque a primera vista parezca alarmante – es exactamente el criterio de honestidad metodológica que el propio checklist declara como su objetivo.

Sobre el conjunto (C10–C15): 4 de las 6 características de conjunto se evalúan como cumplidas con matices (*Complete, Feasible, Comprehensible, Able to be validated*), respaldadas por evidencia directa (los 43 requisitos ya implementados y validados en CI vía `scripts/validate-traceability.sh`). Las otras 2 (*Consistent, Correct*) se marcan explícitamente con excepción: se identificaron 2 inconsistencias reales dentro del conjunto – la contradicción ya autodeclarada entre REQ-F-001/REQ-F-020 sobre el estado inicial de una cuenta tras el registro, y una nueva (encontrada por el propio checklist, no señalada antes) entre las dos cifras de latencia citadas por REQ-NF-003 para la misma condición de caché frío.

Capítulo 5

Materiales y métodos

5.1 Enfoque metodológico: Design Science Research

SGB-SaaS se desarrolló siguiendo, de forma implícita durante el proyecto y reconstruida aquí de forma explícita, la estructura de seis actividades de *Design Science Research* (DSR) de Peffers et al.¹ – un proceso iterativo pensado específicamente para investigación que produce un artefacto (en este caso, software) como resultado, no solo conocimiento descriptivo. Las seis actividades se mapean así contra la evolución real y documentada del proyecto:

1. **Identificación del problema y motivación.** La fricción operativa real de la gestión bibliotecaria manual ([Capítulo 1](#)), delimitada en la fase de planificación del proyecto (Entrega 1A).
2. **Definición de objetivos de una solución.** Las cuatro preguntas de investigación (RQ1–RQ4) y los cinco objetivos específicos (OE1–OE5) declarados en el [Capítulo 1](#).
3. **Diseño y desarrollo.** Construcción incremental del artefacto en sucesivas entregas – Entrega 1B (primer módulo funcional), Entrega 3 (candidato estable con los ocho módulos y evidencia empírica inicial), Entrega Final (v1.0.0, este documento) – con las decisiones de arquitectura formalizadas como ADR y el modelo C4 ([Capítulo 6](#)).
4. **Demostración.** Ejecución de casos de uso reales de punta a punta contra el stack Docker completo, no simulados: `docs/mediciones/backend/2026-07-29-flujo-prestamo-devolucion-multa-e2e.md` y `docs/mediciones/frontend/2026-07-30-flujo-frontent-prestamos-reservaciones-`
5. **Evaluación.** El conjunto de instrumentos de medición descrito en la [Sección 5.2](#) de este mismo capítulo (k6, JaCoCo, Lighthouse, auditoría OWASP, y SUS

¹La formulación de DSR de Peffers et al. (2007), *A Design Science Research Methodology for Information Systems Research*, no está incluida todavía en `docs/bibliografia.bib`: esta tarea recibió instrucción explícita de no agregar referencias nuevas más allá de la excepción de Brooke (1996) para SUS, así que se nombra el enfoque metodológico por autor y año sin `\cite{}` formal. Queda pendiente verificar su DOI real contra Crossref y agregarlo en una tarea futura, con el mismo rigor aplicado al resto de este acervo.

pendiente de ejecución).

6. **Comunicación.** Este mismo informe académico, el checklist de calidad de requisitos INCOSE v4 (`docs/checklists/incose2023-req.md`), la documentación arquitectónica versionada como código (`docs/arquitectura/`), y el artefacto de software archivado con DOI persistente en Zenodo ([Capítulo 1](#)).

5.2 Instrumentos de medición

- **System Usability Scale (SUS)** ([Brooke, 1996](#)): 10 ítems tipo Likert de 5 puntos, alternando afirmaciones positivas y negativas, que producen una puntuación agregada de 0 a 100 por participante mediante la fórmula estándar del instrumento. Es el instrumento del Bloque C.3 de esta guía; su estado de ejecución real – todavía pendiente – se declara con honestidad en la [Sección 5.4](#).
- **k6** (`k6/libros-listado-test.js`, `k6/opts.js`): herramienta de prueba de carga HTTP, usada para medir la latencia del endpoint de catálogo bajo 50 VUs concurrentes en los escenarios `cache_caliente/cache_frio`. Su protocolo completo se describe en la [Sección 5.5](#).
- **Lighthouse** (`docs/mediciones/lighthouse/lhci-20260731-0300.json`): auditoría automatizada de calidad del frontend, con puntuaciones reales obtenidas de 0,95 en Performance, 0,95 en Accessibility, 1,00 en Best Practices y 0,82 en SEO sobre una escala de 0 a 1.
- **JaCoCo** (`jacoco-maven-plugin` en `backend-springboot/pom.xml`): cobertura de línea/rama/complejidad ciclomática sobre el backend, con objetivo declarado de la guía de $\geq 70\%$ de líneas para la Entrega Final. El resultado real vigente es **75,85 %** de líneas (380/501), por encima del umbral, tras un ciclo de trabajo dirigido específicamente a cinco clases de autenticación/autorización que partían de cobertura casi nula (`docs/mediciones/jacoco/2026-07-30-cobertura-jacoco-clases-seguridad-vigente.m`).
- **Auditoría OWASP** (`scripts/owasp-audit.sh`, `docs/mediciones/sec/`): verificación reproducible vía `curl` contra el stack Docker real de un subconjunto representativo de controles del OWASP Top 10:2021 ([Sección 2.7](#)), con hallazgo y corrección documentados por control.

5.3 Enfoque de métricas: Goal-Question-Metric (GQM)

El bloque de evaluación de rendimiento de SGB-SaaS se estructuró, retrospectivamente, según el patrón Goal-Question-Metric:

Meta (Goal). Analizar el uso de caché Redis sobre el endpoint de catálogo, con el propósito de evaluar su efecto sobre el rendimiento observable

(latencia), desde el punto de vista del equipo de desarrollo, en el contexto de carga concurrente controlada (k6, 50 VUs).

Preguntas (*Questions*).

- Q1. ¿Cuál es la latencia p95 del endpoint con caché caliente frente a caché frío?
- Q2. ¿La diferencia observada entre ambos escenarios es estadísticamente significativa, y de qué magnitud es el efecto?
- Q3. ¿Se cumplen los umbrales de latencia exigidos por la guía para cada escenario?

Métricas (*Metrics*).

- M1. Percentiles p50/p90/p95/p99 de `http_req_duration` (ms) por escenario, sobre las 5 corridas agregadas (`scripts/perf-analysis.py`).
- M2. Estadístico y p -valor de la prueba de Wilcoxon de rangos con signo (pareada, sobre el p95 de cada corrida), y tamaño de efecto de Cliff's delta.
- M3. Cumplimiento binario (sí/no) de los umbrales exigidos: p95 < 200 ms en caliente, p95 < 500 ms en frío.

Los resultados reales obtenidos para M1–M3 están en `docs/mediciones/perf/REPORT.md` y se transcriben en el [Capítulo 6](#). Este mismo patrón GQM podría extenderse a los otros bloques de evaluación (seguridad, cobertura, usabilidad); no se desarrolla aquí para cada uno por disciplina de alcance de este capítulo – un GQM completo es suficiente para documentar el enfoque de métricas adoptado, sin redundar en una plantilla repetida para cada bloque.

5.4 Población, muestreo y participantes

La población objetivo real de este proyecto es la comunidad de la Universidad Técnica Estatal de Quevedo (UTEQ) en sus roles de dominio (lector, bibliotecario) – no una muestra sintética ni un panel externo contratado. [Baltes and Ralph \(2022\)](#), en su revisión crítica de prácticas de muestreo en investigación de ingeniería de software, advierten que muestras pequeñas y no aleatorizadas – el perfil esperable de cualquier estudio de este alcance, reclutado por conveniencia dentro de una única institución – limitan severamente la generalización de cualquier conclusión más allá de la muestra concreta estudiada, y recomiendan declarar explícitamente el método de reclutamiento en vez de presentar el resultado como representativo de una población más amplia.

Nota de honestidad – estado real de ejecución. La muestra SUS **no se ha reclutado ni ejecutado todavía** al momento de escribir este capítulo (OBS-08 en `docs/observaciones/OBSERVACIONES.md`: “Pendiente – en progreso, asignada a Panama”). Lo que sí existe, ya definido y versionado antes de la primera ejecución real (no de forma retroactiva), es el protocolo completo: plantilla de consentimiento informado (`docs/etica/consentimientos/plantilla.md`), técnica de muestreo por conveniencia dentro de la comunidad UTEQ, tamaño mínimo de muestra $N \geq 15$ participantes

exigido por la guía, y anonimización mediante código de participante (P01, P02, ..., ver `docs/etica/ETHICS.md`, sección iii) – nunca nombre ni correo. No se declara una muestra ya obtenida para no incurrir en el mismo tipo de afirmación no verificada que este documento evita de forma sistemática en el resto de sus capítulos.

5.5 Protocolo experimental de rendimiento (replicable)

El bloque de rendimiento sigue un protocolo completamente reproducible, documentado con el comando exacto ejecutado:

```
1 docker run --rm --network sgb-saas_default \
2   -v "$(pwd)/k6:/scripts" -v "$(pwd)/docs/mediciones/perf:/out" \
3   grafana/k6 run --out json=/out/k6-run{N}.json /scripts/libros-
   listado-test.js
```

Se ejecutó 5 veces de forma independiente (el mínimo exigido por la guía), cada corrida cubriendo ambos escenarios de forma secuencial (`cache_caliente`: siempre `page=0`, para forzar repetidos aciertos de caché sobre la misma clave; `cache_frio`: página elegida por un PRNG determinista (`mulberry32`, semilla fija 42) en cada petición, para forzar fallo de caché en cada una), con un perfil de carga común de 10s de *ramp-up* a 50 VUs, 30s sostenidos, 10s de *ramp-down* (`k6/opts.js`). Autenticación real vía `POST /api/auth/login` en el `setup()` del script, reutilizando el mismo `accessToken` entre VUs. Los cinco archivos de datos crudos (`k6-run1.json`–`k6-run5.json`, formato NDJSON de k6) están versionados en `docs/mediciones/perf/`, permitiendo reanalizar los mismos datos sin repetir la carga. El detalle completo – entorno exacto (versiones de Docker, Java, PostgreSQL, Redis, k6), resultados por corrida y agregados – está en `docs/mediciones/perf/REPORT.md` y se resume en el [Capítulo 6](#).

5.6 Análisis estadístico

El análisis agregado de las 5 corridas (`scripts/perf-analysis.py`) calcula, por escenario, sobre la métrica `http_req_duration`: la media, la mediana, la desviación típica (σ), el intervalo de confianza al 95 % de la media, los percentiles `p50/p90/p95/p99`, la tasa de error HTTP ≥ 500 y el *throughput* (peticiones/segundo). Sobre la comparación pareada `cache_caliente` vs. `cache_frio` (`p95` de cada una de las 5 corridas, no el *pool* de peticiones agregado, precisamente porque las corridas son las mismas 5 sesiones de carga y no muestras independientes) se aplica la prueba de Wilcoxon de rangos con signo (método exacto, vía `scipy.stats.wilcoxon`) y el tamaño de efecto de Cliff's delta. Los resultados reales obtenidos – Wilcoxon $p = 0,0625$, Cliff's delta = $-0,68$ – y su interpretación correcta en función de la potencia estadística del tamaño de muestra mínimo se desarrollan en el [Capítulo 8](#) (validez de conclusión).

5.7 Determinismo y semillas

Para que los resultados sean reproducibles bit a bit donde el diseño experimental usa aleatoriedad, se fijaron semillas explícitas en los dos únicos puntos del repositorio donde se generan números pseudoaleatorios con fines de medición (verificado por búsqueda directa en el código):

- **Selección de página en el escenario `cache_frio` (`k6/libros-listado-test.js`):** PRNG `mulberry32` con semilla fija **42**, de modo que la secuencia de páginas pedidas en cada corrida es la misma si se re-ejecuta el script.
- **Intervalo de confianza al 95 % por *bootstrap* del percentil p95** en el gráfico comparativo (`scripts/perf-analysis.py`, `BOOTSTRAP_SEED = 42`): 2000 réplicas de remuestreo con reemplazo, con la misma semilla **42** – un percentil, a diferencia de la media, no tiene una fórmula cerrada de intervalo de confianza, de ahí el uso de *bootstrap*.

Ninguna otra parte del repositorio (scripts de prueba, seed de base de datos, generación de datos de ejemplo) usa aleatoriedad con fines de medición que requiera declarar una semilla adicional.

Capítulo 6

Diseño y arquitectura del sistema

El modelo arquitectónico de SGB-SaaS está versionado como código fuente en `docs/arquitectura/workspace.dsl` (Structurizr DSL, modelo C4), en sus tres niveles completos – Contexto, Contenedores y Componentes – que reemplazan los diagramas estáticos sin fuente versionada de la Entrega 1A (`docs/diagramas/diagrama_c4_capa1.png`, `diagrama_c4_capa2.jpeg`).

6.1 Nivel 1 – Contexto del sistema

Cinco actores interactúan con SGB-SaaS como caja negra: **Lector** (consulta el catálogo, reserva libros, ve su historial de préstamos y multas), **Bibliotecario** (registra préstamos, devoluciones y multas desde el mostrador; gestiona el catálogo), **Gerente** (supervisa la operación: cuentas de personal, catálogos maestros de estado y reportes de auditoría), **Administrador** (configura parámetros del sistema y catálogos base de la plataforma – rol que no existía en el diseño original de Entrega 1A, incorporado al normalizar el modelo RBAC) y **Visitante anónimo** (sin cuenta todavía; hoy solo puede registrarse o iniciar sesión).

El propio `workspace.dsl` documenta, en su comentario de cabecera, una discrepancia real entre el diseño original de Entrega 1A y la implementación actual que vale la pena señalar aquí: Gemini 2.0 Flash API y Google Books API aparecían como sistemas externos en el diseño original y se **retiraron** del diagrama actual porque no existe ninguna integración con Google Books en el código real (Gemini sí se integró, pero como parte interna del contenedor Backend, no como un "sistema externo" del Nivel 1 – ver Nivel 3 más abajo). Tampoco se diseñó nunca un endpoint de catálogo público accesible sin autenticación: `LibroController` exige rol `LECTOR` (o superior) incluso para listar libros, así que el alcance real del *Visitante anónimo* se limita a registro e inicio de sesión.

6.2 Nivel 2 – Contenedores

Cuatro contenedores, alineados 1:1 con `docker-compose.yml`:

- **Frontend Angular** (TypeScript/Angular 17) – SPA de catálogo, formularios reactivos y gestión de sesión con el `accessToken` en memoria, nunca en `localStorage`.
- **Backend Spring Boot** (Java 21/Spring Boot 4.0.6) – API REST con autenticación JWT (cookie `HttpOnly` para el `refreshToken`), autorización RBAC por roles, y lógica de negocio de préstamos/reservas/multas vía JPA y procedimientos SQL.
- **PostgreSQL 16** – usuarios, roles/permisos, catálogo, préstamos, reservas, multas y bitácora de auditoría (26 tablas).
- **Redis 7** – blacklist de tokens JWT revocados (logout / revocación) y caché del listado de libros con TTL configurable externamente.

Respecto al diseño original de Entrega 1A, el contenedor Redis y el actor Administrador son incorporaciones directas de la normalización RBAC y del mecanismo de revocación de tokens vía *blacklist*, ambos ausentes del diagrama de contenedores original.

6.3 Nivel 3 – Componentes

El Nivel 3 se completó una vez cerradas las 8 ramas del módulo de Préstamos/Reservas/Multas (evitando así reconstruir el diagrama dos veces mientras ese módulo seguía en desarrollo activo). Se construyó a partir del inventario real de *controllers/services/clases* de seguridad del backend – no del diseño aspiracional –, agrupado por dominio en 21 componentes para mantener el diagrama legible en vez de tener una entrada por cada una de las cerca de 30 clases reales:

- **Capa API** (7 componentes): Auth API, Catálogo API, Préstamos API, Notificaciones API, CredencialQR API, Chatbot API, Admin API – un componente por dominio funcional, no por clase `@RestController` individual.
- **Capa de servicio** (8 componentes): Auth Service, Catálogo Service, Préstamos Service, Reportes Service, Notificaciones Service, CredencialQR Service, Chatbot Service, Admin Service.
- **Seguridad** (5 componentes): `JwtService`, `JwtAuthFilter`, `UserDetailsServiceImpl`, `LoginRateLimiter` (OWASP A07, límite de intentos de login por correo+IP) y `ChatbotRateLimiter` (límite de mensajes al asistente por usuario).
- **Integración externa** (1 componente): `GeminiClient` – cliente HTTP hacia la API de Gemini 2.0 Flash, con reintento simple ante 429/timeout/5xx y sin exponer la URL con la API key en logs (ver ADR-016, [Sección 6.4](#)).

Las relaciones entre componentes se verificaron contra el código real (no se infirieron): cada flecha API → Service, Service → Postgres/ Redis, y las dependencias cruzadas de `ChatbotService` hacia `CatálogoService` y `PréstamosService` para el *grounding* real de sus respuestas (consulta disponibilidad de libros y reservas del propio usuario antes de responder, para no inventar disponibilidad) corresponden a llamadas reales entre las clases Java correspondientes.

Validación sintáctica y exportación visual

El archivo se validó sintácticamente con el motor activamente mantenido `structurizr/structurizr` (subcomando `validate`), no con `structurizr/cli` – que en este momento del ecosistema Structurizr es una imagen retirada que solo imprime un aviso de desuso y termina con código de salida 0 para cualquier entrada, válida o no, por lo que una validación histórica contra esa imagen no habría sido una validación real. El diagrama de Nivel 3 se exportó a SVG mediante el flujo `structurizr/structurizr export -format plantuml` seguido de renderizado con PlantUML, resultando en `docs/arquitectura/c4-nivel3-componentes.svg`, versionado junto al DSL fuente.

6.4 Decisiones arquitectónicas documentadas (ADR)

El repositorio contiene **13 Architecture Decision Records** (`docs/adr/`), agrupados a continuación según los 6 temas obligatorios del Bloque D de la guía (mapeo completo en `docs/adr/README.md`); dos de esos temas están cubiertos por un ADR cuya numeración interna del repositorio no coincide con la numeración sugerida por la guía (ADR-006/ADR-007 sugeridos por la guía para acceso a datos y despliegue corresponden a ADR-013 y ADR-012 en este repositorio), por continuidad con la numeración ya evaluada en la Tercera Entrega – ver la nota explícita en `docs/adr/README.md` sobre por qué no se renumeraron los archivos.

#	Tema obligatorio (Bloque D)	ADR(s)
1	Elección de la pila tecnológica principal	ADR-001
2	Esquema de autenticación	ADR-010 + ADR-003 + ADR-007
3	Gestor de base de datos	ADR-011
4	Estrategia de caché	ADR-008
5	Estrategia de despliegue	ADR-012
6	Estrategia de acceso a datos (CRUD-ORM vs. SP)	ADR-013

Los 7 ADR restantes no mapean a ninguno de los 6 temas obligatorios, pero documentan decisiones reales tomadas durante el desarrollo: ADR-006 (versionado de esquema reproducible, Flyway + snapshot), ADR-009 (licencia MIT), ADR-014 (separación de responsabilidades entre ADMIN y GERENTE: el primero administra permisos/configuración de plataforma, el segundo la operación diaria – ninguno de los dos queda excluido de *ver* el padrón de usuarios, pero solo ADMIN puede *modificarlo*), ADR-015 (TLS termina en el proxy, no en el backend Spring Boot: el backend queda preparado vía `server.forward-headers-strategy: framework` para reconocer X-Forwarded-Proto y activar HSTS el día que el proxy real active TLS, sin cambios de código adicionales), y ADR-016 (qué datos exactos viajan a la API externa de Gemini en el chatbot – texto del mensaje, historial de la sesión y un prompt de *grounding* con datos ya públicos del catálogo – y por qué eso no constituye una exposición indebida de información: nunca viajan credenciales, tokens, contraseñas ni identificación personal).

6.5 Atributos de calidad (ISO/IEC 25010)

`docs/arquitectura/ISO25010.md` prioriza las 8 características de calidad de producto de la norma para el contexto específico de este proyecto (biblioteca universitaria de bajo volumen, no un sistema de alta concurrencia), citando evidencia real donde existe en vez de estimarla. Se reproduce a continuación tal cual el documento fuente (única fuente de verdad de esta tabla; este capítulo no la duplica de memoria):

Característica	Prioridad	Estrategia / decisión que la atiende
Adecuación funcional	Alta	Procedimientos almacenados transaccionales para operaciones con lógica cruzada multi-tabla (<code>sp_registrar_devolucion</code> , <code>sp_pagar_multa</code> , <code>sp_anular_multa</code>), constraints de integridad referencial (FKs, CHECKs) en <code>db/schema.sql</code> .
Eficiencia de desempeño	Media	Caché Redis del listado de libros con TTL externo (ADR-008). Evidencia vigente: prueba de carga formal (50 VUs, 5 corridas de k6 con Wilcoxon + Cliff's delta) – p95 agregado 19,49 ms en <code>cache_caliente</code> (umbral <200 ms) y 7,50 ms en <code>cache_frio</code> (umbral <500 ms), 0 % de errores 5xx en 20 003 peticiones (<code>docs/mediciones/perf/REPORT.md</code>). El propio informe declara una limitación metodológica real (ver Capítulo 8): <code>cache_frio</code> mide sobre todo consultas con página vacía, no el costo real de traer una página llena sin caché. Una medición manual preliminar del 21 de julio había reportado cifras de otro orden de magnitud (~170 ms/~30 ms) por tratarse de una sola petición aislada sin carga concurrente; queda solo como antecedente histórico, no como evidencia vigente.
Compatibilidad	Media	API REST documentada con OpenAPI/Swagger vía <code>springdoc-openapi</code> (ADR-001), JSON como formato estándar – sin integración real hoy con sistemas académicos institucionales ni con Google Books API.
Usabilidad	Alta	<code>sp_registrar_devolucion</code> valida el estado del préstamo antes de actuar (rechaza con LB409 si ya estaba devuelto, evitando duplicidad); frontend Angular con formularios reactivos y validación antes de enviar.
Fiabilidad	Alta	Parcialmente atendida: ADR-003 documenta explícitamente que, si Redis cae, <code>JwtAuthFilter</code> queda sin forma de verificar revocaciones – la decisión <i>fail-open</i> vs. <i>fail-closed</i> sigue anotada como pendiente para producción, no resuelta. Los <i>healthchecks</i> de <code>docker-compose.yml</code> sí garantizan un orden de arranque correcto.

Característica	Prioridad	Estrategia / decisión que la atiende
Seguridad	Alta	Cookie <code>HttpOnly+Secure+SameSite=Strict</code> para el <code>refreshToken</code> (ADR-007), BCrypt costo 12, blacklist de tokens revocados (ADR-003), respuestas de error RFC 7807 sin fuga de detalles internos.
Mantenibilidad	Alta	Los ADR de <code>docs/adr/</code> documentan decisiones no obvias con sus alternativas consideradas; <code>docs/basedatos/CATALOGO-SP.md</code> documenta el criterio de separación CRUD-ORM vs. procedimientos. Nota de este informe¹ : la cifra real de ADRs es 13, no 6 – ver aclaración al pie.
Portabilidad	Alta	Docker Compose con imágenes base pinadas por digest sha256, <code>db/init/01-consolidado.sql</code> generado de forma reproducible, arranque de un solo comando vía <code>make up</code> .

6.6 Matriz de trazabilidad – resumen por tipo de acceso a datos

La matriz completa de los 43 requisitos vive en `docs/trazabilidad/matriz.csv` y se valida automáticamente en cada ejecución de CI vía `scripts/validate-traceability.sh`. Este capítulo no reproduce las 43 filas (el detalle completo por requisito vive en [Capítulo 4](#) y en la propia matriz); resume aquí la columna `tipo_acceso`, relevante para esta sección por ser la evidencia directa de que la estrategia híbrida CRUD-ORM/SP declarada en ADR-013 se aplica de verdad y no solo en teoría.

Nota de corrección respecto a una versión anterior de este capítulo. `matriz.csv` tenía 4 filas (REQ-F-018, REQ-F-019, REQ-F-020, REQ-F-028) con una coma sin escapar dentro de un campo, que rompía el parseo CSV estándar – defecto ya corregido en el commit `ecfaf52`. Una versión anterior de esta tabla, calculada antes de esa corrección, solo había detectado 2 de las 4 filas afectadas (REQ-F-019, REQ-F-020) y las marcaba como “sin clasificar”; además atribuía el problema de REQ-F-019 a la columna `tipo_acceso`, cuando en realidad ese campo sí tenía un valor de `tipo_acceso` válido (“CRUD-ORM + generacion de imagen (ZXing)”) – el campo roto en esa fila específica era `modulo_codigo`, no `tipo_acceso`. Con las 43 filas parseando limpio, la tabla de abajo reclasifica las 4 filas correctamente en vez de dejarlas fuera:

¹El texto original de `IS025010.md` en esta fila cita “6 ADRs existentes”, cifra que quedó desactualizada tras incorporar los 8 módulos construidos después de la Tercera Entrega – el repositorio contiene 13 ADRs a la fecha de este capítulo ([Sección 6.4](#)). Se transcribe el texto original de la fuente junto con esta aclaración en vez de editarlo silenciosamente, ya que corregir `IS025010.md` está fuera del alcance de esta tarea.

Tipo de acceso a datos	Requisitos	%
CRUD-ORM (exclusivo, incl. combinado con Redis/SMTP/caché/API externa)	21	48,8 %
Híbrido SP + CRUD-ORM (requisitos transversales)	2	4,7 %
Procedimiento/función SQL (SP, exclusivo, incl. combinado con generación de PDF)	8	18,6 %
No aplica (decisión de arquitectura, configuración, control de acceso o UI sin acceso directo a BD)	11	25,6 %
Redis + SMTP (sin tabla Postgres nueva; no aplica CRUD-ORM ni SP) ²	1	2,3 %
Total	43	100,0 %

De los 43 requisitos, 10 (23,3 %) involucran al menos un procedimiento o función SQL (8 exclusivamente, 2 en combinación con CRUD-ORM: REQ-NF-004, transversal a los módulos de préstamos/multas/reservaciones vs. auth/libros/bitácora, y REQ-NF-013, prevención de inyección SQL sobre ambos mecanismos), consistente con la justificación de ADR-013: reservar los procedimientos almacenados para operaciones con validación cruzada multi-tabla o transacciones atómicas compuestas (registrar préstamo, registrar devolución, pagar multa, anular multa, expirar reservaciones vencidas, y los 3 reportes agregados), dejando el CRUD elemental de una sola tabla en Spring Data JPA. REQ-F-018 (renovación de préstamo), pese a mencionar textualmente “SP” en su valor de `tipo_acceso` (“CRUD-ORM (validación en Java, sin SP nuevo)”), es CRUD-ORM exclusivo: la frase aclara explícitamente que la renovación *no* introdujo un procedimiento almacenado nuevo, la validación de sus 3 reglas de negocio corre en Java – se señala aquí porque una primera pasada de clasificación automática por coincidencia de texto la contó por error como híbrida, hasta revisar el valor completo del campo.

²REQ-F-020: `tipo_acceso` = “Redis (código OTP con TTL, sin tabla nueva) + SMTP (EmailService)”. Categoría propia, no un defecto de la matriz – el código de verificación vive en Redis con TTL, sin persistirse en una tabla Postgres nueva, y el envío usa SMTP directo; no encaja limpiamente en CRUD-ORM ni en SP.

Capítulo 7

Implementación

Este capítulo documenta decisiones críticas de implementación del backend, cada una con al menos un listado de código real extraído directamente del repositorio.

7.1 Procedimiento almacenado y su invocación desde Spring Data JPA

La estrategia híbrida de acceso a datos (ADR-013, [Sección 6.4](#)) exige que las operaciones con validación cruzada multi-tabla se implementen como procedimiento o función SQL. El siguiente listado es `db/procs/sp_crear_prestamo.sql` completo: registra un préstamo de forma atómica, validando que el usuario no esté bloqueado por multas y que el libro tenga stock disponible, usando SQLSTATE personalizados (LB404/LB422) para que el backend traduzca el error sin parsear texto.

Listing 7.1: `db/procs/sp_crear_prestamo.sql` (completo)

```
1 CREATE OR REPLACE FUNCTION sp_crear_prestamo(  
2     p_usuario_id BIGINT,  
3     p_libro_id BIGINT,  
4     p_bibliotecario_id BIGINT,  
5     p_dias_prestamo INTEGER  
6 )  
7 RETURNS BIGINT  
8 LANGUAGE plpgsql  
9 AS $$  
10 DECLARE  
11     v_estado_usuario_id      INTEGER;  
12     v_estado_bloqueado_id    INTEGER;  
13     v_stock_disponible       SMALLINT;  
14     v_estado_activo_prestamo_id INTEGER;  
15     v_prestamo_id            BIGINT;  
16 BEGIN  
17     SELECT id INTO v_estado_bloqueado_id  
18         FROM estados_usuario  
19         WHERE nombre = 'BLOQUEADO_POR_MULTA';
```

```

20
21     SELECT estado_id INTO v_estado_usuario_id
22     FROM usuarios
23     WHERE id = p_usuario_id
24     FOR UPDATE;
25
26     IF NOT FOUND THEN
27         RAISE EXCEPTION 'El usuario % no existe', p_usuario_id
28         USING ERRCODE = 'LB404';
29     END IF;
30
31     IF v_estado_usuario_id = v_estado_bloqueado_id THEN
32         RAISE EXCEPTION 'El usuario % esta bloqueado por multas
33         pendientes ...',
34         p_usuario_id USING ERRCODE = 'LB422';
35     END IF;
36
37     SELECT stock_disponible INTO v_stock_disponible
38     FROM libros WHERE id = p_libro_id FOR UPDATE;
39
40     IF NOT FOUND THEN
41         RAISE EXCEPTION 'El libro % no existe', p_libro_id
42         USING ERRCODE = 'LB404';
43     END IF;
44
45     IF v_stock_disponible <= 0 THEN
46         RAISE EXCEPTION 'El libro % no tiene stock disponible',
47         p_libro_id
48         USING ERRCODE = 'LB422';
49     END IF;
50
51     UPDATE libros SET stock_disponible = stock_disponible - 1
52     WHERE id = p_libro_id;
53
54     INSERT INTO prestamos (usuario_id, libro_id,
55     bibliotecario_id,
56     fecha_prestamo, fecha_devolucion_estimada,
57     estado_prestamo_id)
58     VALUES (p_usuario_id, p_libro_id, p_bibliotecario_id, NOW(),
59     NOW() + make_interval(days => p_dias_prestamo),
60     v_estado_activo_prestamo_id)
61     RETURNING id INTO v_prestamo_id;
62
63     RETURN v_prestamo_id;
64 END;
65 $$;

```

Nota de honestidad sobre el mecanismo de invocación. La guía de esta entrega espera que el método Java invoque el procedimiento con la anotación `@Procedure` de Jakarta Persistence. En este repositorio, ese *fue* el mecanismo original, pero se reemplazó por `@Query(nativeQuery = true)` tras un fallo real en tiempo

de ejecución: Hibernate genera sintaxis de parámetros nombrados de PostgreSQL (nombre =>valor) dentro del escape JDBC {call ...} usado por @Procedure, que el driver pgjdbc no soporta ahí – PrestamoMultiProcedureIntegrationTest (test de integración real contra PostgreSQL, no mocks) confirmó el fallo en la primera ejecución. La migración de los 5 métodos afectados está documentada en docs/mediciones/backend/2026-07-28-fallo-invocacion-sp-multi-out.md y referenciada como *issue* conocido de Spring Data JPA (spring-projects/spring-data-jpa#3393). El siguiente listado muestra el método real vigente hoy en PrestamoProcedureRepository.java, incluyendo el comentario del propio código que documenta por qué ya no usa @Procedure:

Listing 7.2: PrestamoProcedureRepository.spCrearPrestamo (mecanismo vigente)

```

1  /**
2   * sp_crear_prestamo: retorno escalar unico (BIGINT). Antes
3   * usaba
4   * @Procedure(procedureName=...) -- fallaba en runtime con
5   * "syntax error at or near '=>'" porque Hibernate genera
6   * sintaxis de
7   * parametros nombrados de PostgreSQL dentro del escape JDBC
8   * {call ...}, que pgjdbc no soporta. @Query nativa con SELECT
9   * evita
10  * el CallableStatement por completo.
11  */
12 @Query(value = "SELECT sp_crear_prestamo(:p_usuario_id, :
13   p_libro_id, " +
14   ":p_bibliotecario_id, :p_dias_prestamo)",
15   nativeQuery = true)
16 Long spCrearPrestamo(
17   @Param("p_usuario_id") Long usuarioId,
18   @Param("p_libro_id") Long libroId,
19   @Param("p_bibliotecario_id") Long bibliotecarioId,
20   @Param("p_dias_prestamo") Integer diasPrestamo
21 );

```

Los parámetros siguen bindeándose por nombre (:p_usuario_id, etc.) exactamente igual que con @Procedure – la única diferencia es el mecanismo JDBC que usa Hibernate para transportar la llamada (PreparedStatement en vez de CallableStatement), por lo que la prohibición de SQL dinámico o concatenación de entrada de usuario (regla A.2.3 de la guía) se sigue cumpliendo igual (docs/basedatos/CATALOGO-SP.md, sección “Nota de diseño: por qué 5 usan @Procedure...y 2 usan @Query”). De los 7 objetos SQL del catálogo, los 5 con efectos secundarios usaban originalmente @Procedure/@NamedStoredProcedureQuery y hoy usan @Query(nativeQuery = true) tras esta migración; las 2 funciones de solo lectura que retornan TABLE (varias filas) usaron @Query nativa desde el principio, porque la API de *stored procedures* de JPA 2.1 no tiene una forma estándar de exponer un RETURNS TABLE de PostgreSQL a través de @Procedure.

¿Esto viola la prohibición de SQL dinámico de la guía (A.2.1)?

No. La guía (Bloque A.2.1) establece textualmente que “queda prohibido invocarlos mediante concatenación de cadenas en `createNativeQuery(...)`”. Esa prohibición apunta a un patrón concreto y distinto del usado en este proyecto: construir la sentencia SQL en tiempo de ejecución **concatenando texto de entrada del usuario** dentro del propio literal de la consulta – p. ej. `createNativeQuery("SELECT sp_crear_prestamo(" + idUsuario + ")")`, donde `idUsuario` pasa a formar parte literal de la cadena SQL antes de ejecutarse, el vector clásico de inyección SQL.

El código real de este proyecto **no hace eso en ningún punto**. El listado 7.2 usa `@Query(value = "SELECT sp_crear_prestamo(:p_usuario_id, ...)", nativeQuery = true)` con **parámetros nombrados bindeados** (`:p_usuario_id`, `:p_libro_id`, ...) resueltos por Spring Data JPA/Hibernate vía `PreparedStatement` antes de llegar al driver JDBC – el valor de cada parámetro nunca se concatena, formatea ni interpola dentro del literal de la cadena SQL; el literal que Java compila es fijo (`"SELECT sp_crear_prestamo(:p_usuario_id, :p_libro_id, :p_bibliotecario_id, :p_dias_prestamo)"`) y el binding ocurre por separado, exactamente como con `@Procedure`. Esta forma de `@Query` nativo con parámetros nombrados es, en sí misma, un mecanismo formal y documentado de Spring Data JPA para invocar funciones/procedimientos parametrizados – no es un atajo informal ni una forma disimulada de SQL dinámico.

En síntesis: **la regla A.2.1 prohíbe construir SQL concatenando entrada de usuario dentro del texto de la consulta; los parámetros bindeados de `@Query` nativa no construyen SQL a partir de input en absoluto, por lo que este patrón no cae bajo esa prohibición**. El motivo del cambio desde `@Procedure`/`@NamedStoredProcedureQuery` tampoco fue estilístico: está documentado con evidencia real y verificable en `docs/mediciones/backend/2026-07-28-fallo-invocacion-sp-multi-out.md` (fallo reproducido en runtime, con el mensaje de error real de PostgreSQL/pgjdbc) y corresponde a un defecto conocido de la combinación Hibernate 6.2+/7.x con procedimientos multi-OUT en PostgreSQL, reportado en el repositorio público de Spring Data JPA (`spring-projects/spring-data-jpa#3393`) – no una decisión arbitraria del equipo para eludir el mecanismo que pide la guía.

Nota sobre verificación automática. Al momento de escribir este capítulo no existe todavía en el repositorio un script de análisis estático tipo `scripts/audit-sql-dynamic.sh` que detecte patrones de SQL dinámico en el código fuente – se deja constancia aquí para que, cuando se incorpore, su regla de detección distinga explícitamente entre concatenación real de entrada de usuario dentro del texto de una consulta (el patrón que sí debe marcar) y `@Query(nativeQuery = true)` con parámetros nombrados bindeados (`:p_...`) como el de esta sección, que no debería marcarse como hallazgo – un análisis ingenuo basado solo en buscar la cadena `nativeQuery = true` sin inspeccionar si el valor de `value` contiene variables Java concatenadas produciría un falso positivo contra este patrón, ya verificado como seguro.

7.2 Manejo uniforme de errores (RFC 7807)

Todas las respuestas de error del backend se devuelven como `ProblemDetail` (RFC 7807), vía un único `@RestControllerAdvice` (`GlobalExceptionHandler`). Un caso particularmente representativo es la traducción de los `SQLSTATE` personalizados de los procedimientos almacenados (Sección 7.1) a códigos HTTP, sin que el backend tenga que parsear el texto del mensaje de error:

Listing 7.3: `GlobalExceptionHandler.handleStoredProcedureError` (extracto real)

```
1 @ExceptionHandler({
2     InvalidDataAccessResourceUsageException.class,
3     UncategorizedSQLException.class,
4     DataAccessException.class
5 })
6 public ProblemDetail handleStoredProcedureError(Exception ex) {
7     Throwable causa = ex;
8     while (causa != null && !(causa instanceof SQLException)) {
9         causa = causa.getCause();
10    }
11
12    if (causa instanceof SQLException sqlEx) {
13        String sqlState = sqlEx.getSQLState();
14        HttpStatus status = switch (sqlState) {
15            case "LB404" -> HttpStatus.NOT_FOUND;
16            case "LB409" -> HttpStatus.CONFLICT;
17            case "LB422" -> HttpStatus.UNPROCESSABLE_ENTITY;
18            default -> null;
19        };
20        if (status != null) {
21            return ProblemDetail.forStatusAndDetail(status,
22                sqlEx.getMessage());
23        }
24
25        log.error("Error no controlado en procedimiento almacenado",
26            ex);
27        return ProblemDetail.forStatusAndDetail(
28            HttpStatus.INTERNAL_SERVER_ERROR, "Error interno del
29            servidor");
30    }
31 }
```

Sin este manejador dedicado, cualquier `RAISE EXCEPTION ... USING ERRCODE = 'LBxxx'` de un procedimiento llegaba envuelto en una `DataAccessException` genérica de Spring, y caía en el *catch-all* de último recurso – un error de negocio real (p. ej. “préstamo ya devuelto”) se veía como un 500 genérico en vez de un 409 con el mensaje real. El mismo patrón de un `@ExceptionHandler` dedicado por tipo de excepción se repite para excepciones de dominio Java (p. ej. `CorreoYaRegistradoException` → 409, `LoginRateLimitExcedidoException` → 429) y para excepciones del propio framework que antes caían también en el *catch-all*: `AuthorizationDeniedException` (autorización de método vía `@PreAuthorize`,

antes producía 500 en vez de 403) y `NoResourceFoundException` (recurso estático inexistente bajo el perfil `prod`, corregido en esta misma entrega – ver `docs/mediciones/sec/2026-08-11-owasp-a05-verificacion-real.md`).

7.3 Configuración paramétrica del sistema

`ConfiguracionSistemaService` expone parámetros de la tabla `configuracion_sistema` (p. ej. el máximo de renovaciones de un préstamo, ver `REQ-F-018`) para leerse en cada operación de negocio sin ir a la base de datos cada vez, con un caché en memoria simple (`ConcurrentHashMap` por instancia, no Redis) invalidado clave por clave al escribir:

Listing 7.4: `ConfiguracionSistemaService` (extracto real: lectura cacheada y escritura)

```
1 private final ConcurrentHashMap<String, String> cache = new
    ConcurrentHashMap<>();
2
3 @Transactional(readOnly = true)
4 public String obtenerValor(String clave) {
5     String cacheado = cache.get(clave);
6     if (cacheado != null) {
7         return cacheado;
8     }
9     String valor = repo.findById(clave)
10         .orElseThrow(() -> new EntityNotFoundException(
11             CLAVE_NO_ENCONTRADA + clave))
12         .getValor();
13     cache.put(clave, valor);
14     return valor;
15 }
16
17 @Transactional
18 public ConfiguracionSistemaResponseDTO actualizar(String clave,
19     String nuevoValor) {
20     ConfiguracionSistema config = repo.findById(clave)
21         .orElseThrow(() -> new EntityNotFoundException(
22             CLAVE_NO_ENCONTRADA + clave));
23     config.setValor(nuevoValor);
24     repo.save(config);
25     cache.remove(clave);
26     return new ConfiguracionSistemaResponseDTO(config.getClave(),
27         config.getValor());
28 }
```

La elección deliberada de un `ConcurrentHashMap` local en vez de Redis (que sí se usa para el caché del catálogo, ver [Sección 7.4](#)) está documentada en el propio Javadoc de la clase: este valor cambia con muy poca frecuencia (un ADMIN ajustando un parámetro puntual), a diferencia del caché “libros”, que sí necesita compartirse entre instancias e

invalidarse por TTL. Además, **actualizar** solo permite modificar claves **ya existentes** – el catálogo de parámetros válidos lo define el dominio (migraciones/*seed*), no quien llama al endpoint, evitando que quede una clave suelta sin ningún consumidor real.

7.4 Estrategia de caché Redis del catálogo

El listado de libros usa caché declarativo de Spring (`@Cacheable/@CacheEvict`) sobre Redis, con TTL configurable externamente (ADR-008):

Listing 7.5: LibroService (extracto real: lectura cacheada y evicción en escritura)

```
1 @Cacheable("libros")
2 @Transactional(readOnly = true)
3 public Page<LibroResponseDTO> listar(Pageable pageable) {
4     return libroRepo.findByEstado_Nombre(ESTADO_ACTIVO, pageable
5         )
6         .map(this::toDTO);
7 }
8 @CacheEvict(value = "libros", allEntries = true)
9 @Transactional
10 public LibroResponseDTO crear(LibroRequestDTO dto) {
11     if (libroRepo.existsByIsbn(dto.isbn())) {
12         throw new IllegalArgumentException("ISBN ya registrado:
13             " + dto.isbn());
14     }
15     validarStock(dto.stockTotal(), dto.stockDisponible());
16     return toDTO(libroRepo.save(fromDTO(dto)));
17 }
```

`crear`, `actualizar` y `eliminar` invalidan el caché completo del listado (`allEntries = true`) para no dejar una respuesta paginada obsoleta tras cualquier escritura. El método `listarPorCategoria` (filtro por categoría/autor) deliberadamente **no** lleva `@Cacheable`: combinar el filtro con el caché exigiría una clave compuesta fuera del alcance de la rama que lo introdujo, y no es el camino más transitado del catálogo – decisión documentada como comentario en el propio código, no un descuido. El método `sugerir` (autocompletado) usa un caché *distinto* ("`sugerencias-libros`") con TTL propio, mucho más corto (5–10 s), porque su patrón de invalidación (una entrada por texto de búsqueda) es incompatible con el caché de listado general.

7.5 Transporte del token de sesión en cookie `HttpOnly`

El `refreshToken` viaja exclusivamente en una cookie `HttpOnly`, `Secure`, `SameSite=Strict`, con `path=/api/auth` (ADR-007) – nunca en el cuerpo JSON, para que JavaScript del frontend no pueda leerlo ni reenviarlo manualmente:

Listing 7.6: `AuthController.buildRefreshCookie` (real, completo)

```
1 private ResponseCookie buildRefreshCookie(String valor, long
   maxAgeMs) {
2     return ResponseCookie.from(REFRESH_COOKIE_NAME, valor)
3         .httpOnly(true)
4         .secure(true)
5         .sameSite("Strict")
6         .path("/api/auth")
7         .maxAge(Duration.ofMillis(maxAgeMs))
8         .build();
9 }
```

El mismo método se reutiliza para *limpiar* la cookie en `logout` (`buildRefreshCookie("", 0)`, valor vacío y `maxAge=0`). El `accessToken`, de vida corta (1h), sigue viajando en el cuerpo JSON de la respuesta – migrarlo también a cookie está deliberadamente diferido (ver la nota de honestidad de ADR-007 y REQ-NF-002, [Capítulo 4](#)) por el impacto que tendría en `jwt.interceptor.ts/auth.service.ts` del frontend, no por un olvido.

Capítulo 8

Amenazas a la validez

Este capítulo analiza, en las 4 categorías estándar de amenazas a la validez de un estudio empírico de ingeniería de software, las limitaciones **reales** ya documentadas en el repositorio – no una lista genérica de amenazas de relleno que no correspondan a algo efectivamente observado o declarado durante este proyecto.

8.1 Validez de constructo

La amenaza de constructo más concreta de este proyecto está en el propio diseño del experimento de rendimiento (Sección 7.4, docs/mediciones/perf/REPORT.md): el escenario `cache_frio` se diseñó para medir el costo de una consulta al catálogo *sin* caché (el constructo de interés, para contrastarlo contra `cache_caliente`), pero su implementación real – un PRNG que elige una página aleatoria entre 0 y 999 en cada petición – hace que la gran mayoría de esas páginas queden fuera de rango de la tabla `libro` (poblada con datos de ejemplo de desarrollo, no con miles de registros), de modo que Spring Data devuelve una `Page` vacía sin necesidad de traer filas reales. En la práctica, `cache_frio` mide sobre todo el costo de una consulta `COUNT + SELECT` con resultado vacío, **no** el costo real de traer y serializar una página completa de libros sin caché – una construcción operacional distinta de la que el nombre del escenario sugiere. La comparación “caché caliente vs. caché frío” de este experimento, por tanto, **no aísla limpiamente el efecto del caché**: parte de la diferencia de latencia observada entre ambos escenarios refleja también el costo distinto de las consultas que cada uno ejecuta, no únicamente la presencia o ausencia de caché. Esta limitación está documentada explícitamente en el propio `REPORT.md` (punto 3 de su “Análisis breve”), no descubierta recién en este capítulo – se reproduce aquí porque es exactamente el tipo de amenaza de constructo que este capítulo debe declarar, en vez de reemplazarla por una amenaza genérica que no corresponda al proyecto real.

8.2 Validez interna

La corrida 1 de las 5 corridas de k6 ([Capítulo 6](#), `docs/mediciones/perf/REPORT.md`) presenta una cola de latencia anómala en `cache_caliente` (p95=107,31 ms, frente a un rango de 6,25–12,36 ms en las 4 corridas siguientes) que no se repite en ninguna corrida posterior. La explicación documentada es *warm-up* de JVM/JIT y del *pool* de conexiones de HikariCP: la corrida 1 fue la primera ejecución de carga real contra un contenedor `sgb_backend` que llevaba varias horas sin tráfico significativo. Esto constituye una variable extraña (*confound*) no controlada directamente por el diseño experimental – no se descartó la corrida ni se repitió hasta obtener un resultado más limpio (lo que habría sido, en sí mismo, una amenaza distinta: sesgo de selección de datos), sino que se documentó y se incluyó en el agregado de las 5 corridas, confirmando además mediante el test estadístico pareado que el patrón `caliente > frio` se sostiene igual si se considera solo el subconjunto de las corridas 2–5. Una amenaza de validez interna distinta, propia de la auditoría de seguridad ([Capítulo 6](#)), es que el mismo equipo que implementó el sistema es quien ejecutó y documentó la auditoría OWASP: no existe una segunda parte independiente que haya intentado explotar el sistema sin conocimiento previo de su código fuente, lo que puede subestimar la superficie real de vulnerabilidades frente a una prueba de penetración por un equipo externo.

8.3 Validez externa

La amenaza de validez externa más relevante y todavía sin resolver es el tamaño de muestra de la evidencia de usabilidad (System Usability Scale, SUS): la guía de esta entrega exige un mínimo de $N \geq 15$ participantes, y esa evidencia sigue **pendiente** al momento de escribir este capítulo (OBS-08 en `docs/observaciones/OBSERVACIONES.md`, “en progreso, asignada a Panama”, sin commit real todavía). El protocolo completo – plantilla de consentimiento informado, tarea de *onboarding* de dos pasos, código de participante anónimo – ya está definido y validado, pero no ejecutado; este capítulo no puede, por tanto, evaluar la validez externa del resultado SUS en sí (todavía no existe), solo señalar el riesgo de tamaño de muestra que enfrentará cuando se ejecute. Ese riesgo no es menor: [Baltes and Ralph \(2022\)](#) documentan, en su revisión crítica de prácticas de muestreo en investigación de ingeniería de software, que muestras pequeñas y no aleatorizadas – exactamente el perfil esperable de un estudio SUS de PFC con participantes reclutados por conveniencia dentro de la comunidad universitaria – limitan severamente la generalización de cualquier conclusión más allá de la muestra concreta estudiada, y recomiendan reportar explícitamente el método de reclutamiento y sus límites en vez de presentar el resultado como representativo de una población más amplia sin justificarlo. Cuando la muestra SUS se ejecute, este capítulo deberá actualizarse citando el tamaño real obtenido, el método de reclutamiento, y aplicando ese mismo criterio de [Baltes and Ralph \(2022\)](#) a la interpretación del puntaje. Una segunda amenaza de validez externa, más estructural: todos los hallazgos de este proyecto provienen de un único contexto (una biblioteca universitaria ecuatoriana de bajo volumen, con un equipo de 3 personas sin dedicación exclusiva) – ninguna conclusión aquí debería generalizarse sin más a bibliotecas de otro tipo (públicas de gran escala, especializadas, con perfiles de carga o de usuario distintos).

Una tercera amenaza de validez externa, distinta de las dos anteriores, es el alcance del propio acervo bibliográfico reunido para la revisión de trabajos relacionados ([Capítulo 3](#)): de las 33 referencias que lo componen – cada una verificada individualmente contra el registro oficial de Crossref (autor, DOI y venue reales; ese mismo proceso de verificación detectó y corrigió 5 errores de atribución de fuente durante su construcción, evidencia del rigor con que se aplicó) – solo **12 califican como de “alto impacto”** en el sentido estricto que exige el criterio D6 de la guía (revista con cuartil JCR Q1/Q2, o conferencia CORE A del tipo ICSE/FSE/ASE/MSR/EASE/ESEM), por debajo del umbral de 20 que exige el nivel “Excelente” de la rúbrica – aunque sí se cumple el umbral de 30 referencias totales. Esto no refleja un déficit de esfuerzo de búsqueda: el dominio concreto de este PFC (gestión bibliotecaria universitaria con IA aplicada) es un nicho de aplicación con un volumen de publicación considerablemente menor en los venues de primer nivel de Ingeniería de Software que otras áreas más centrales de la disciplina (pruebas de software, arquitectura, DevOps); la literatura realmente disponible sobre este dominio se concentra, en cambio, en revistas especializadas de ciencias de la información (típicamente Q2) y en conferencias regionales o temáticas – reales e indexadas, pero fuera del conjunto CORE A que domina la producción de Ingeniería de Software general. Frente a esta limitación real y ya verificada de forma exhaustiva, la decisión tomada fue reportar el número exacto obtenido (12 de 33) en vez de completar el conteo con citas de venues de calidad no verificable – el mismo criterio de verificación cruzada contra Crossref que corrigió los 5 errores de atribución mencionados arriba es, en sí mismo, la evidencia de que ese criterio se aplicó de forma consistente en todo el acervo, no solo donde convenía a un conteo más favorable.

8.4 Validez de conclusión

La comparación estadística pareada entre `cache_caliente` y `cache_frio` (Wilcoxon de rangos con signo sobre el p95 de cada una de las 5 corridas, `scripts/perf-analysis.py`) es el ejemplo más concreto de amenaza a la validez de conclusión de este proyecto: el estadístico exacto de dos colas con $n = 5$ pares tiene un valor mínimo alcanzable de exactamente $p = 0,0625$ cuando las 5 diferencias apuntan en la misma dirección sin excepción – que es precisamente lo que ocurre aquí (`cache_caliente > cache_frio` en las 5 corridas) –, por lo que el resultado **no alcanza el umbral convencional de significancia** ($p < 0,05$) pese a un tamaño de efecto grande y consistente (Cliff’s delta = $-0,68$). Tratar este resultado como “no hay diferencia” sería una conclusión estadísticamente incorrecta: el límite real es la potencia estadística del tamaño de muestra mínimo exigido por la guía ($n = 5$ corridas), no evidencia sustantiva en contra del efecto observado – más corridas subirían la potencia sin que se espere que cambie la dirección de la conclusión, ya consistente en las 5 disponibles. Esta interpretación se documenta de forma explícita en `docs/mediciones/perf/REPORT.md` en vez de reportar solo el p -valor sin su contexto de potencia, que por sí solo induciría a la conclusión errónea de ausencia de efecto. Una segunda amenaza de conclusión, ya señalada en la sección de validez de constructo de este mismo capítulo, es que ese mismo resultado ($p = 0,0625$, efecto grande) se interpreta sobre una comparación que no aísla limpiamente el constructo de interés (el efecto del caché en sí) – ambas amenazas se refuerzan mutuamente y deben leerse juntas, no de forma aislada, para no

sobre-interpretar la magnitud exacta del efecto del caché reportada aquí.

Bibliografía

- Adetayo, A. J. (2023). Artificial intelligence chatbots in academic libraries: the rise of ChatGPT. *Library Hi Tech News*, 40(3):18–21.
- Ahmed, S. and Mahmood, Q. (2019). An authentication based scheme for applications using JSON web token. In *2019 22nd International Multitopic Conference (INMIC)*, pages 1–6. IEEE.
- Aienobe, V. and Iqbal, M. Z. (2025). Responsive web design for enhanced user experience (UX) and user interface (UI). *International Journal of Computer Applications*, 187(34):72–93.
- Ayemowa, M. O., Ibrahim, R., and Khan, M. M. (2024). Analysis of recommender system using generative artificial intelligence: A systematic literature review. *IEEE Access*, 12:87742–87766.
- Ayinde, L., Ebiefung, R., and Oladokun, B. D. (2026). Adoption of artificial intelligence in academic libraries: A systematic review of current practices, challenges, and research opportunities. *The Journal of Academic Librarianship*, 52(1):103185.
- Baltes, S. and Ralph, P. (2022). Sampling in software engineering research: a critical review and guidelines. *Empirical Software Engineering*, 27(4):94.
- Bano, M., Zowghi, D., Ferrari, A., Spoletini, P., and Donati, B. (2019). Teaching requirements elicitation interviews: an empirical study of learning from mistakes. *Requirements Engineering*, 24:259–289.
- Bass, L., Clements, P., and Kazman, R. (2021). *Software Architecture in Practice*. Addison-Wesley, 4th edition.
- Brooke, J. (1996). SUS: A “quick and dirty” usability scale. In Jordan, P. W., Thomas, B., Weerdmeester, B. A., and McClelland, I. L., editors, *Usability Evaluation in Industry*, pages 189–194. Taylor & Francis, London.
- Brown, S. (2023). The C4 model for visualising software architecture. Leanpub. <https://c4model.com/>. Recurso técnico autopublicado, no arbitrado por pares; se cita como referencia normativa de notación, no como evidencia de investigación empírica.
- Bucko, A., Vishi, K., Krasniqi, B., and Rexha, B. (2023). Enhancing JWT authentication and authorization in web applications based on user behavior history. *Computers*, 12(4):78.
- Colley, D., Stanier, C., and Asaduzzaman, M. (2018). The impact of object-relational mapping frameworks on relational query performance. In *2018 International Confe-*

- rence on Computing, Electronics & Communications Engineering (iCCECE), pages 47–52. IEEE.
- Ehrenpreis, M. and DeLooper, J. P. (2022). Implementing a chatbot on a library website. *Journal of Web Librarianship*, 16(2):120–142.
- Fowler, M. (2002). *Patterns of Enterprise Application Architecture*. Addison-Wesley.
- Hu, P. and Zhang, Y. (2025). Big data analytics in library services with AI: Personalized content recommendations and catalog optimization. *IEEE Access*, 13:88412–88420.
- Kurtanović, Z. and Maalej, W. (2017). Automatically classifying functional and non-functional requirements using supervised machine learning. In *2017 IEEE 25th International Requirements Engineering Conference (RE)*, pages 490–495. IEEE.
- Kusuma, M., Widyawan, and Ferdiana, R. (2017). Performance comparison of caching strategy on WordPress multisite. In *2017 3rd International Conference on Science and Technology - Computer (ICST)*. IEEE.
- Liabor, V. G. R. (2023). Enhancing library services through digital cataloging techniques. *International Journal of Advanced Research in Science, Communication and Technology*, pages 516–526.
- McKay, K. A. and Cooper, D. A. (2019). Guidelines for the selection, configuration, and use of transport layer security (TLS) implementations. NIST Special Publication 800-52 Rev. 2, National Institute of Standards and Technology.
- More, N. S. (2024). Intelligent library management system. *Trends in Computer Science and Information Technology*, 9(1):001–009.
- Mukund, R. N., Arun, S., Kusuma, S. M., Vishak, K. R., and Monisha, M. (2021). Intelligent RFID based library management system. In *2021 IEEE International Conference on Electronics, Computing and Communication Technologies (CONECCT)*, pages 1–6. IEEE.
- Ng, T.-J., Ng, K.-W., and Haw, S.-C. (2024). Lib-bot: A smart librarian-chatbot assistant. *International Journal of Computing and Digital Systems*, 15(1):1–11.
- Palomares, C., Franch, X., Quer, C., Chatzipetrou, P., López, L., and Gorschek, T. (2021). The state-of-practice in requirements elicitation: an extended interview study at 12 companies. *Requirements Engineering*, 26:273–299.
- Parlakkılıç, A. (2022). Evaluating the effects of responsive design on the usability of academic websites in the pandemic. *Education and Information Technologies*, 27(1):1307–1322.
- Rahman, A., Indrajit, E., Unggul, A., and Dazki, E. (2024). Agile project management impacts software development team productivity. *Sinkron*, 8(3):1847–1858.
- Rajačić, T., Boberić-Krstićev, D., Tešendić, D., and Milosavljević, G. (2025). Validation of GDPR compliance in a library management system: A BISIS and TeMDA case study. *Information Technology and Libraries*, 44(4).
- Shatnawi, A. S., Al-Duwairi, B., and Samarneh, A. A. (2024). Comprehensive empirical study of Python JWT libraries. *Procedia Computer Science*, 238:827–832.

- Sommerville, I. (2011). *Software Engineering*. Addison-Wesley, 9th edition.
- Supriyono, H., Fitriyan, M. R., and Muamaroh (2018). Developing a QR code-based library management system with case study of private school in surakarta city indonesia. In *2018 Third International Conference on Informatics and Computing (ICIC)*, pages 1–6. IEEE.
- Truong, D. M., Xu, L., and de Vrieze, P. T. (2025). The impact of personality traits on scrum team effectiveness: Insights from vietnamese software development companies. *Information and Software Technology*, 188:107878.
- Walls, C. (2022). *Spring in Action*. Manning Publications, 6th edition.
- Wayesa, F., Leranso, M., Asefa, G., and Kedir, A. (2023). Pattern-based hybrid book recommendation system using semantic relationships. *Scientific Reports*, 13:3693.
- Yan, R., Zhao, X., and Mazumdar, S. (2023). Chatbots in libraries: A systematic literature review. *Education for Information*, 39(4):431–449.
- Yu, E. S. K. (1997). Towards modelling and reasoning support for early-phase requirements engineering. In *Proceedings of ISRE '97: 3rd IEEE International Symposium on Requirements Engineering*, pages 226–235. IEEE.